

California Lutheran University

Predictive Modeling for Direct Marketing Optimization in the Financial Services

ECON 562: Analytics II
John Garcia
Spring 2025

Shaun Levenson
5-18-2025

Predictive Modeling for Direct Marketing Optimization in the Banking Sector

Introduction

In today's competitive financial services environment, banks aim to enhance customer engagement and increase revenue through targeted direct marketing campaigns. Our client, a major financial institution, seeks to better understand customer behavior to drive the adoption of new products and services. Using a dataset of over one million customer records, this project develops three predictive models: a classification model to determine product adoption (B_TGT), and two numeric models to predict total new sales (INT_TGT) and the number of new products purchased (CNT_TGT). Following the CRISP-DM methodology, we conduct thorough exploratory data analysis, feature engineering, and model selection using H2O's automatic machine learning function to identify the most important predictors and most accurate predictive modeling techniques. The performance of these models on the test set will inform future marketing strategies and guide long-term consulting decisions for the client.

Overview

This project aims to build predictive models that help a major financial institution better understand and anticipate customer behavior in response to marketing efforts. Specifically, we focus on three target variables: B_TGT, a binary indicator of whether a customer tried a new product; INT_TGT, representing total new sales per customer; and CNT_TGT, the count of new products purchased. To support both strategic planning and actionable targeting, our goal is to develop models that balance predictive accuracy with interpretability.

We use H2O's AutoML function to efficiently train and evaluate multiple modeling techniques—including GLM, DRF, GBM, and Deep Learning—across both classification and

regression tasks. By extracting the top models from each method, we ensure a comprehensive assessment of performance across different algorithm families. The resulting models are evaluated using appropriate metrics for each target variable, laying the foundation for data-driven marketing strategies and future consulting recommendations.

Data Background

The dataset used in this analysis comprises **over 1 million observations (1,060,038 rows)** and includes **24 variables** encompassing identifiers, target outcomes, and a wide range of customer-level predictors. It is structured to support predictive modeling across multiple outcomes related to customer purchasing behavior. Specifically, the dataset is partitioned into three subsets – training, validation, and testing – which facilitates robust model training and evaluation.

There are **three primary target variables** of interest. `b_tgt` is a **binary indicator** representing whether a customer has tried a new product. `int_tgt` is a **continuous numeric variable** that captures the **total new sales** made to each customer. `cnt_tgt` is a **count variable** indicating the **number of new products and services** a customer has purchased. The predictors include both **categorical** and **numeric** inputs, such as customer demographics, behavioral metrics, and purchasing history.

The **numeric predictors** (18 variables) primarily consist of **RFM metrics** (Recency, Frequency, Monetary value), along with demographic features like `demog_age`, `demog_homeval`, `demog_inc`, and `demog_pr`. Categorical predictors (6 variables) include account activity level, customer value segment, gender, and homeownership status. A unique identifier (account) ensures traceability at the individual level, and a partition flag (dataset) indicates the data split used for modeling, however I ended up removing this in favor of an `h2o`

Spring 2025

partition. This rich and well-labeled dataset provides a comprehensive foundation for developing predictive models tailored to customer segmentation, targeting, and product uptake. Full descriptions of the variables can be found in **Appendix.a** in **Table 1** and **Table 2**.

Data Quality Check

As part of the initial data quality checks, I conducted a comprehensive assessment of missing values and structural integrity in the dataset. Four variables were identified with missing values: `int_tgt`, `cnt_tgt`, `rfm3`, and `demog_age`. The percentage of missing values for each was relatively low and is documented in a summary table. For the variable `int_tgt`, which represents total new sales, missing values were found only in cases where `b_tgt` was 0—indicating the customer did not purchase a product. In this context, imputing missing `int_tgt` values with 0 was a logical decision, as it clearly reflects a lack of purchase activity and avoids the unnecessary removal of valid records.

For the other variables—`rfm3`, `cnt_tgt`, and `demog_age`—I opted to remove observations with missing values. This decision was driven by the project goal of producing a highly interpretable model for the client. Introducing imputation methods for these predictors could obscure the interpretability of variable relationships, which is particularly important for stakeholders in a financial setting. Notably, only one observation had a missing value in `cnt_tgt`, so its exclusion had negligible impact. Additionally, I removed any records where `demog_age` was less than 16. This threshold was applied based on legal and practical considerations, as customers under 16 typically lack financial autonomy. These steps ensured that the data passed structural and validity checks, laying a clean and reliable foundation for subsequent exploratory analysis. Shown in **Table 3** are the exact numbers and percentage shares of missing values.

Table 3: Missing Values

<i>Variable</i>	<i># Missing</i>	<i>% Missing</i>
<i>int_tgt</i>	848529	80.0
<i>demog_age</i>	266861	25.2
<i>rfm3</i>	225786	21.3
<i>cnt_tgt</i>	1	0

As part of the data quality checks, all variables initially stored as character types were systematically converted to factors. This step ensures that categorical information is correctly treated in downstream analyses and modeling, preserving the integrity of level-based distinctions. Converting character variables to factors was essential for maintaining accurate representations of categorical data and enabling proper handling by modeling algorithms that rely on factor structures. All other important DQC factors seemed to be in order.

Exploratory Data Analysis

Summary Statistics

After addressing missing values, several numerical variables exhibited substantial changes in their distribution. Most notably, the range of *int_tgt* decreased dramatically, with the maximum dropping from 500,000 to 200,000. This reflects the removal of extreme outliers and the imputation of 0s for non-purchasing customers, resulting in a large decrease in the mean (from 11,235.87 to 2,583.73) and skewness (from 13.30 to 4.77). Similarly, *rfm3* saw a major reduction in skewness from 114.92 to 9.85, indicating a more balanced distribution after excluding rows with missing values. Changes in central tendency were also observed across several variables—for example, *demog_inc* increased in mean (from 40,368.69 to 46,562.05) and median, suggesting that the removed cases tended to have lower incomes. Overall, these shifts

highlight how missing data handling can significantly affect the interpretation of key variables and influence modeling outcomes.

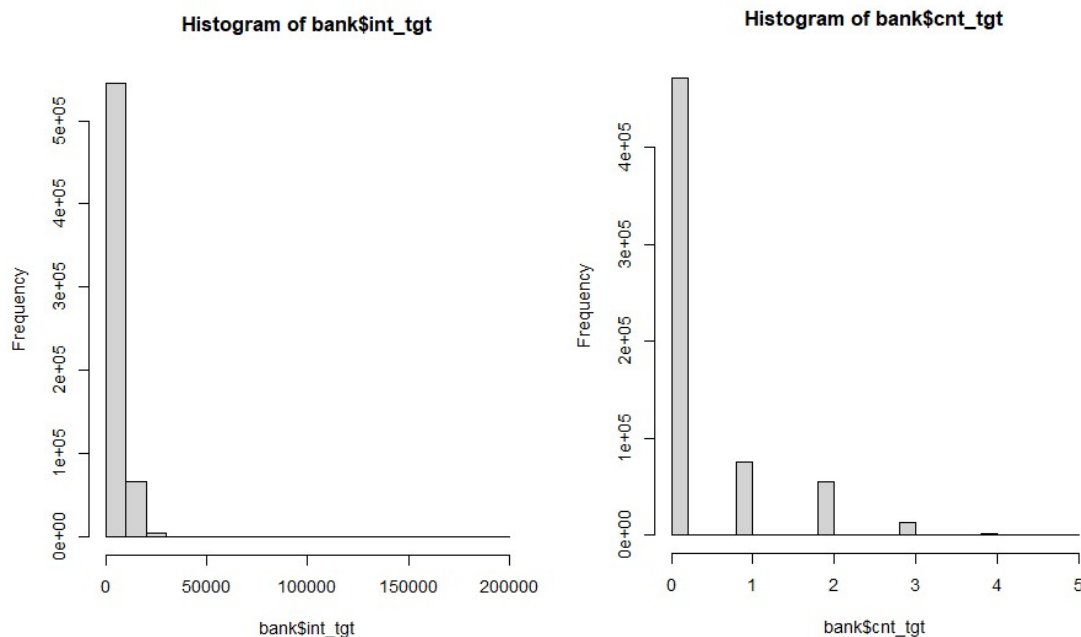
Table 4 and 8: Conjoined Summary Statistics – Numerical

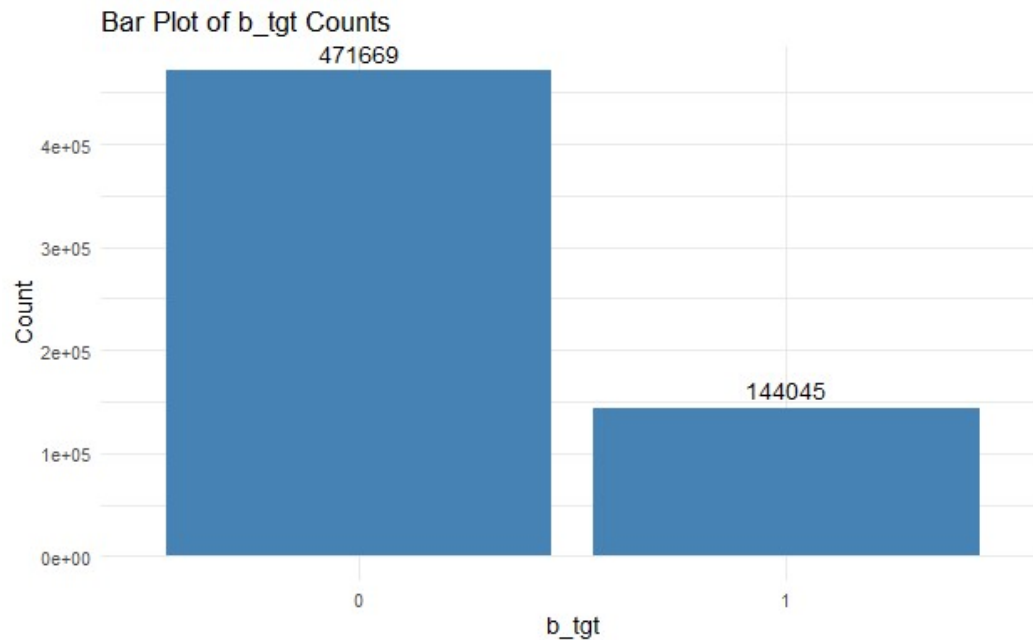
Variable	Stage	Mean	Std Dev	Median	Min	Max	Skewness
<i>int_tgt</i>	Before	11235.87	8491.80	10000.00	0.00	500000.00	13.30
–	After	2583.73	5931.40	0.00	0.00	200000.00	4.77
<i>cnt_tgt</i>	Before	0.31	0.70	0.00	0.00	6.00	2.40
–	After	0.37	0.76	0.00	0.00	5.00	2.09
<i>rfm1</i>	Before	16.09	19.30	15.00	0.00	3713.31	103.31
–	After	15.44	15.09	13.98	1.15	2507.50	84.12
<i>rfm2</i>	Before	13.35	9.47	11.67	1.58	650.00	11.12
–	After	12.80	8.93	11.10	1.58	436.00	9.92
<i>rfm3</i>	Before	15.31	18.97	14.00	0.00	3713.31	114.92
–	After	15.22	11.10	14.00	0.00	600.00	9.85
<i>rfm4</i>	Before	17.47	37.55	15.00	0.00	10000.00	207.18
–	After	16.70	44.24	15.00	0.00	10000.00	205.74
<i>rfm5</i>	Before	2.91	2.03	2.00	0.00	18.00	1.23
–	After	3.29	2.05	3.00	0.00	18.00	1.10
<i>rfm6</i>	Before	9.54	8.47	7.00	0.00	127.00	1.91
–	After	10.34	8.78	8.00	0.00	127.00	1.77
<i>rfm7</i>	Before	1.67	1.53	1.00	0.00	11.00	1.22
–	After	2.07	1.46	2.00	0.00	11.00	1.24
<i>rfm8</i>	Before	5.03	4.51	4.00	0.00	46.00	1.42
–	After	5.67	4.61	4.00	0.00	36.00	1.31
<i>rfm9</i>	Before	18.35	4.02	18.00	2.00	29.00	-0.60
–	After	18.18	3.87	18.00	2.00	28.00	-0.73
<i>rfm10</i>	Before	12.89	4.61	12.00	0.00	77.00	2.86
–	After	13.07	4.51	13.00	0.00	60.00	2.91
<i>rfm11</i>	Before	5.36	1.36	6.00	0.00	22.00	0.32
–	After	5.50	1.29	6.00	0.00	17.00	0.47
<i>rfm12</i>	Before	68.13	37.35	64.00	0.00	571.00	0.30
–	After	69.32	37.36	65.00	13.00	571.00	0.30
<i>demog_age</i>	Before	58.72	16.85	60.00	-1.00	89.00	-0.36
–	After	59.11	16.43	60.00	16.00	89.00	-0.24
<i>demog_homeval</i>	Before	106103.55	93289.97	73880.00	0.00	600067.00	2.46
–	After	106619.64	92456.21	74908.00	0.00	600067.00	2.50
<i>demog_inc</i>	Before	40368.69	28029.02	43174.00	0.00	200007.00	0.23
–	After	46562.05	25211.91	46450.00	0.00	200007.00	0.24
<i>demog_pr</i>	Before	30.57	11.53	31.00	0.00	101.00	-0.15
–	After	31.16	11.24	31.00	0.00	100.00	-0.09

Handling missing values did not significantly alter the distribution of categorical variables, with the exception of `demog_ho`, which became noticeably more imbalanced following imputation.

Target Variable Analysis

This section explores the three target variables used in the analysis. `B_TGT` is a binary variable indicating whether an individual tried a new product (Yes or No). `INT_TGT` is a numeric variable representing the total value of new product sales made by the individual. `CNT_TGT` is a count variable reflecting the number of new products purchased. Together, these variables offer complementary perspectives on customer behavior, capturing participation, quantity, and monetary value. Shown below are their respective distributions.





The three graphs illustrate the distribution of the target variables used in the analysis and provide important insights into the structure of the dataset.

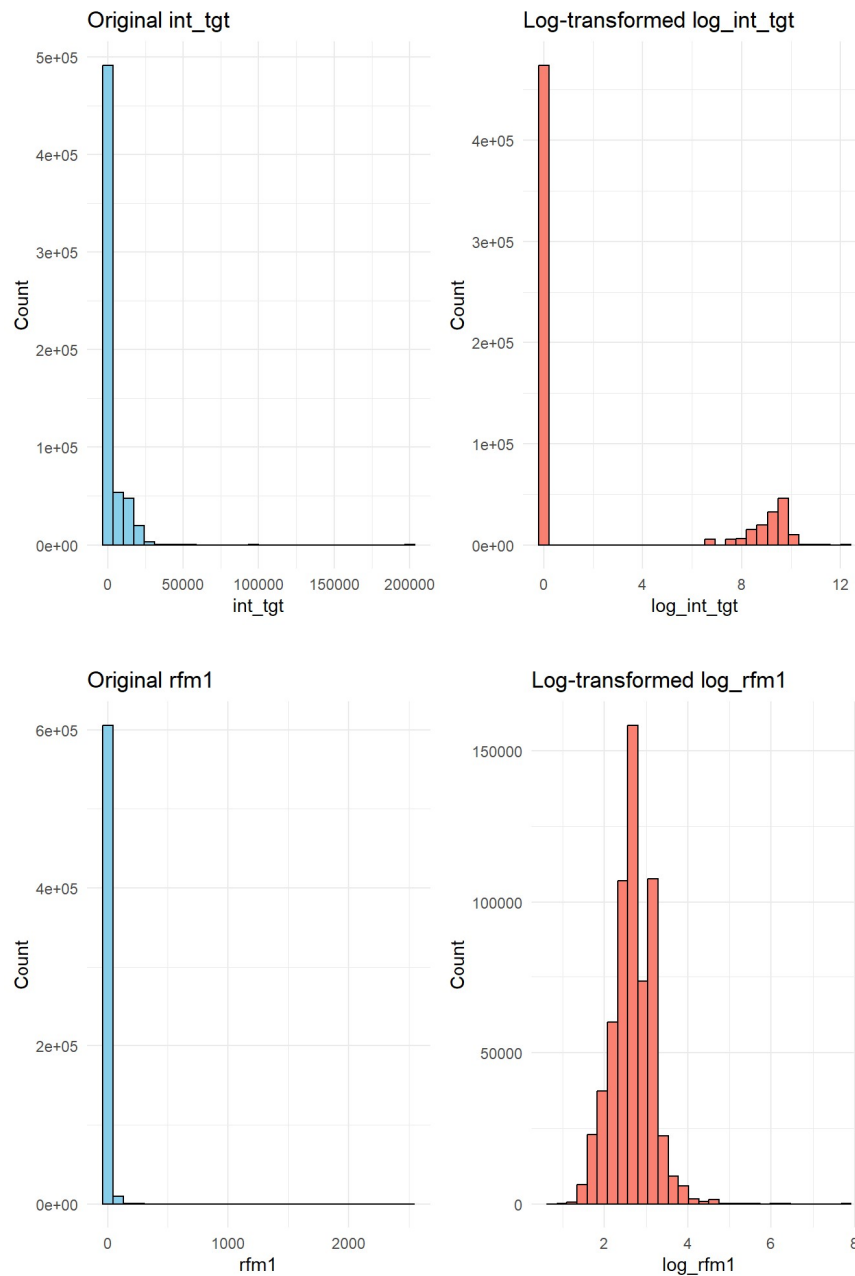
The first histogram shows the distribution of **int_tgt**, representing total new sales. The distribution is heavily right-skewed, with the vast majority of observations concentrated near zero and a small number of extreme values upwards of 200000 (shown by summary statistics). This pronounced skew suggests that a transformation, such as a $\log(x + 1)$ scaling, is appropriate to reduce the influence of outliers and improve normality for modeling purposes, which is further depicted by the skew of 4.77.

The second histogram displays **cnt_tgt**, which counts the number of new products purchased. Similar to **int_tgt**, this variable is also strongly right-skewed. Most customers purchased zero or one new product, and only a few purchased more than that. The discrete nature and skewness of this variable indicate that count-based models, such as Poisson or negative binomial regression, could be more suitable for prediction tasks involving **cnt_tgt**.

The third bar chart presents the distribution of **b_tgt**, a binary indicator of whether a customer tried a new product. The chart reveals a significant class imbalance, with roughly 80% of customers not trying a new product and only about 20% doing so. This imbalance should be taken into account when modeling, potentially using resampling techniques or model strategies that account for class weights to avoid bias toward the majority class. However, due to the clients requests of focusing on causality for marketing purposes, using strategies such as SMOTE may cause biased coefficients for regression based models.

Transformations

As part of the exploratory data analysis, I applied log transformations to several highly skewed numeric variables, specifically int_tgt, rfm1, rfm2, rfm3, rfm4, and rfm10. The transformation used was $\log(x + 1)$, which helps to handle zero values while reducing skewness in the data distributions while maintaining a level of interpretability that transformations such as box-cox do not provide. This adjustment is critical because many machine learning algorithms perform better and produce more stable results when the input variables approximate a normal distribution. Additionally, the log transformation facilitates more meaningful interpretation of model coefficients, allowing them to be understood in terms of relative or percentage changes rather than absolute differences. Summary statistics for the transformed variables were computed and compared to their original counterparts to confirm the effect of the transformation on data distribution. Shown below in the chart space are before and after histograms of int_tgt and rfm1 as examples, showing that performing transformations significantly improves normality.

Chart 9: Transformation Visualization

I also put consideration towards transforming `demog_homeval`, however using a log transformation for this variable flipped the skew to the other side and increased it's magnitude, thus harming normality assumptions.

Variable Correlations

The correlation analysis reveals several strong relationships among the numerical predictors, particularly within the RFM variables. Notably, rfm1, rfm2, and rfm3 exhibit extremely high correlations, with values reaching as high as 0.96. This is intuitive given that these variables capture related measures of sales and promotional responses over different time frames, leading to considerable overlap in the information they provide. Similarly, rfm6 and rfm8 show a strong correlation of 0.89, reflecting the close connection between total lifetime purchases and those specifically driven by direct promotions. Additionally, rfm5 demonstrates a moderate to strong correlation with the target variable cnt_tgt (0.79), underscoring its potential value as a predictor for modeling customer purchase behavior.

Table 6: Strong Correlations ($> |0.6|$)

Variable1	Variable2	Correlation
rfm3	rfm1	0.96
rfm2	rfm1	0.91
rfm8	rfm6	0.89
rfm3	rfm2	0.88
rfm7	rfm5	0.80
rfm5	cnt_tgt	0.79
rfm4	rfm2	0.76
rfm12	rfm8	0.74
rfm12	rfm6	0.71
rfm4	rfm1	0.69
rfm4	rfm3	0.66
rfm11	rfm10	0.66
rfm7	cnt_tgt	0.62

While strong correlations among predictors such as **rfm1**, **rfm2**, and **rfm3** raised concerns about multicollinearity, I leveraged the **h2o AutoML** framework to address these challenges effectively. The AutoML function performs automatic variable selection, coefficient regularization, and dimensionality reduction during model training. This helps to mitigate multicollinearity by shrinking less important coefficients and selecting the most relevant

features, thereby improving model stability and interpretability without requiring manual intervention. Although alternative approaches like Principal Component Analysis (PCA) or explicit removal of highly correlated variables are commonly used to handle multicollinearity, the AutoML approach provides a streamlined and data-driven solution that balances predictive performance with model simplicity. Additionally, leaving in these highly correlated variables may allow for more information to be extracted.

Interactions

To capture more complex, non-additive relationships between key demographic variables, I created three interaction terms. The interaction between income and home value (`demog_inc_homeval`) aims to reflect combined economic capacity, where high income and high home value together may signal greater financial potential than either alone. Similarly, the age and home value interaction (`demog_age_homeval`) may capture generational differences in wealth accumulation and property ownership – older individuals with higher-value homes might exhibit different spending behavior than younger individuals with similar assets. Finally, the interaction between age and income (`demog_age_inc`) considers life stage effects; for instance, younger individuals earning a high income may behave differently from older individuals with the same income. These interactions help enrich the feature set by introducing potentially meaningful nonlinear relationships that could enhance model performance.

Model Fitting

As previously mentioned several times, I took full advantage of h2o's automatic machine learning algorithms to smooth modelling efforts and optimize work time. AutoML automates the supervised machine learning model training process by finding the best model,

given a training dataframe and a response, which then returns a leaderboard of the best performing models that were trained in the process. The models were ranked by their corresponding performance metric (RMSE for `int_tgt` and `cnt_tgt`, AUC for `b_tgt`). The AutoML function has access to the following algorithms: DRF (Distributed Random Forest, an ensemble learning algorithm that builds multiple decision trees in parallel for robust, scalable predictions), GLM (Generalized Linear Model, which allows for the modelling of continuous, binary, and count response variables), XGBoost (fast, scalable machine learning algorithm based on gradient-boosted decision trees), GBM (Gradient Boosting Machine, an ensemble method that builds trees sequentially, optimizing performance by correcting errors from previous trees), Deep Learning (A form of neural network), and Stacked Ensemble (which stacks the performances of many different models to account for each algorithms individual issues). The models that I considered specifically for each were DRF, GBM, Deep Learning, and GLM to get a wide range of predictability and interpretability.

For consistency and to maximize the information extracted from the data, I used the same set of variables across all models. The only exceptions were perfectly multicollinear variables—such as **demog_genf** and **demog_genm**, where I retained only **demog_genf**—and identifiers like **account**, which do not contribute meaningful predictive value.

Model Comparison and Final Evaluation

Modelling – int_tgt

The target metric used for **int_tgt** was RMSE. As shown in **Table 10**, DRF and GBM both vastly outperformed compared to Deep Learning and GLM, with DRF in this case taking rank 1 with an RMSE of 1.4412 (lower RMSE is better) after completing validation. As shown in **Table**

11, running that model on the holdout test set returns highly comparable results, indicating that this model is very generalizable and runs well on unseen data.

Table 10: Int_tgt Validation

Model Rank	Type	RMSE	MSE	Unlogged_RMSE	r2
<i>Model 1</i>	drf	1.4412	2.0770	2.64	0.8599
<i>Model 2</i>	gbm	1.7095	2.9224	4.19	0.8029
<i>Model 3</i>	deeplearning	3.1637	10.0093	149.10	0.3251
<i>Model 4</i>	glm	3.3184	11.0119	246.15	0.2575

Table 11: Best Model

Model Rank	Type	RMSE	MSE	Unlogged_RMSE	r2
<i>Final Model (Model 1)</i>	drf	1.4457	2.0902	2.66	0.8585

Modelling – b_tgt

The target metric used for **b_tgt** was AUC (Area Under Curve). As shown in **Table 13**, DRF and GBM both vastly outperformed compared to Deep Learning and GLM once again, with GBM in this case taking rank 1 with an exceptional AUC of 0.9970 (higher AUC is better) after completing validation. As shown in **Table 14**, running that model on the holdout test set returns highly comparable results, indicating that this model is very generalizable and runs well on unseen data. A visualization of the model performance can be seen in **Appendix.a** in **Chart 6**.

Table 13: b_tgt Validation

Model Rank	Type	AUC
<i>Model 1</i>	gbm	0.9970
<i>Model 2</i>	drf	0.9961
<i>Model 3</i>	deeplearning	0.8657
<i>Model 4</i>	glm	0.8521

Table 14: Best Model b_tgt

Model Rank	Type	AUC
<i>Best Model (Model 1)</i>	gbm	0.9965

Modelling – cnt_tgt

The target metric used for **cnt_tgt** was RMSE. As shown in **Table 17**, DRF and GBM both vastly outperformed compared to Deep Learning and GLM, with DRF in this case taking rank 1 with an RMSE of 0.2574 (lower RMSE is better) after completing validation. As shown in **Table 18**, running that model on the holdout test set returns highly comparable results, indicating that this model is very generalizable and runs well on unseen data.

Table 17: cnt_tgt Validation

<i>Model</i>	<i>Type</i>	<i>RMSE</i>	<i>MSE</i>
<i>Model 1</i>	drf	0.2574	0.0663
<i>Model 2</i>	gbm	0.2754	0.0758
<i>Model 3</i>	deeplearning	0.5745	0.3301
<i>Model 4</i>	glm	0.6165	0.3800

Table 18: Best Model

<i>Model</i>	<i>Type</i>	<i>RMSE</i>	<i>MSE</i>
<i>Best Model (Test Set)</i>	drf	0.2562	0.0656

Causal Factors

The drawback of using the models shown above is that picking out factor effects can be a little bit tricky. However, we can leverage Variable Importance Plots as well as the signs on the coefficients for the GLM model to incorporate a broad, robust interpretation of the variables.

Looking at **Charts 3, 5, and 7** provides relative importance for each variable in the models shown above.

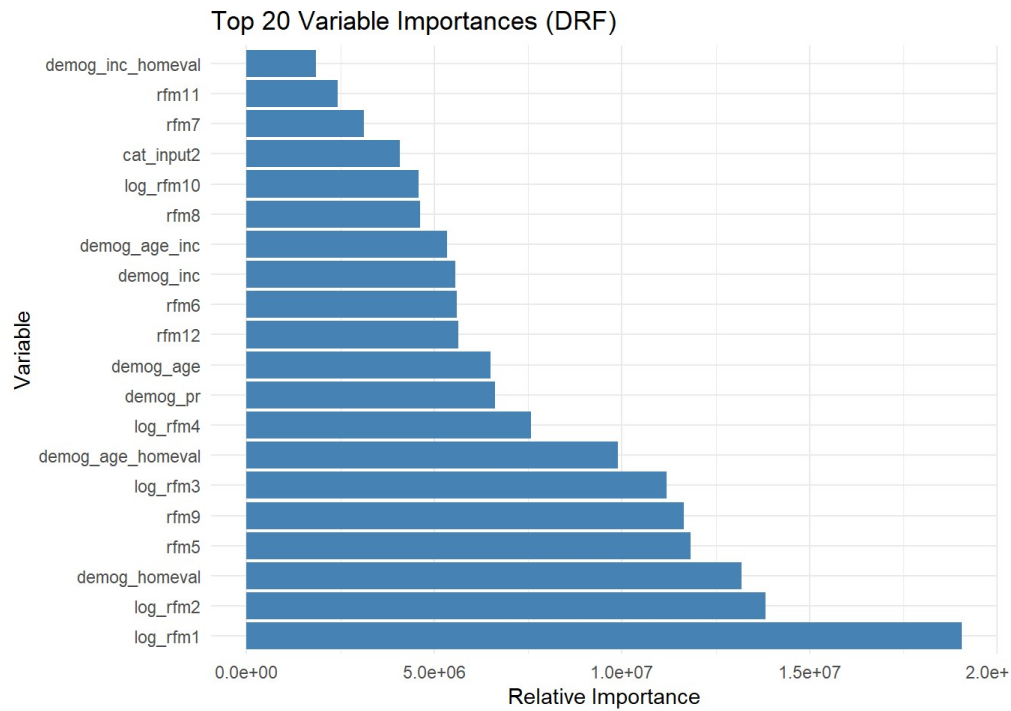
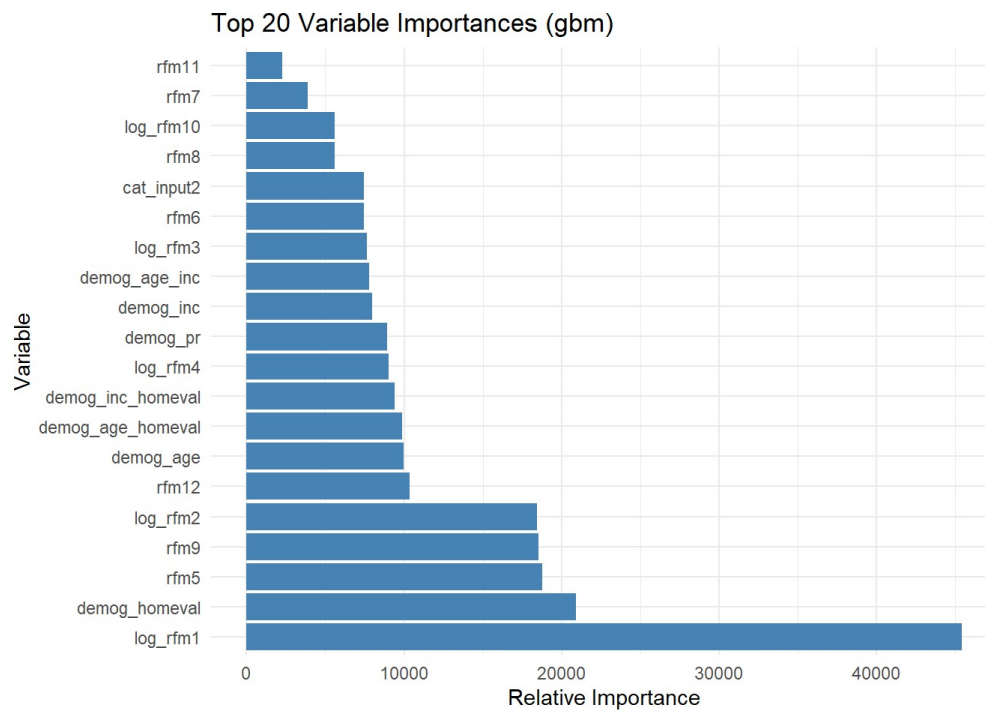
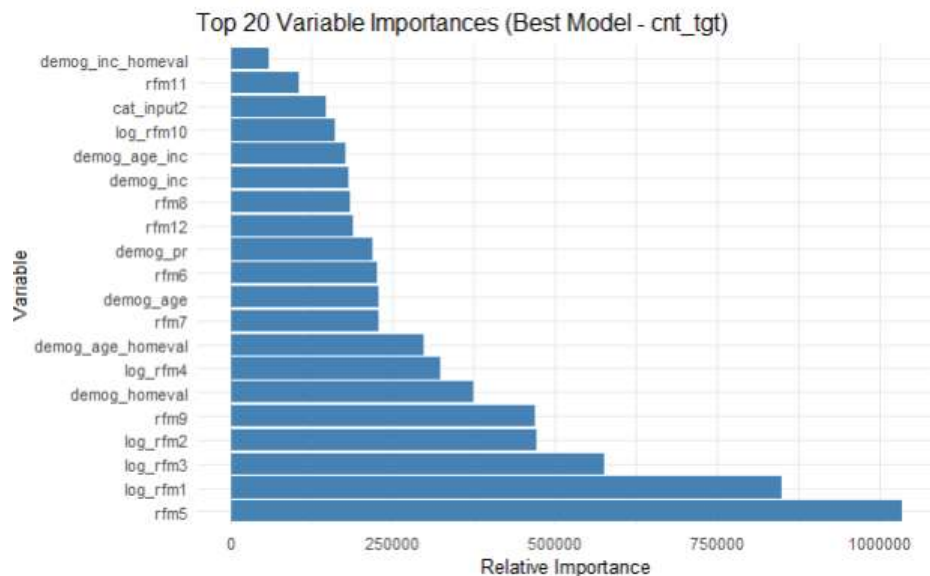
Chart 3: Variable Importance for Int_tgt**Chart 5: Variable Importance for b_tgt**

Chart 7: Variable Importance cnt_tgt

Notice that we consistently see variables such as rfm1, rfm2, and demog_homeval as the variables with the most impact in the models. This is absolutely to be expected as rfm1 and 2 are variables regarding average sales (past 3 years and lifetime respectively) and demog_homeval (home value) can be used as a proxy for wealth, where logically an individual with greater wealth is more likely to spend more on financial products/services. Interestingly, the coefficients for rfm1 across all response variables is negative, indicating that as average sales increase from previous years, total new sales, trying new products, and buying more of them decrease. This is likely due to market saturation, in which these customers have already spent a large amount on products in the past, thus decreasing the likelihood of purchasing more in the future. This can be useful for marketing purposes as marketing towards those who have not yet spent large quantities on products may be more willing to try them out if they were made aware of the new products.

This is in contrast to rfm5 (count purchased past 3 years), which had a consistent positive impact across responses. This may indicate that, unlike average sales, these customers

are consistent and loyal, likely indicating high repeats, making them great candidates for marketing. These individuals would likely be more responsive to direct promotions and great deals.

When considering marketing campaigns, you should also take into consideration what sort of individuals you should avoid marketing towards, so as to not waste resources. Rfm9 (months since last purchase), is an incredibly important factor to consider for marketing. It has a large negative coefficient and is highly impactful across models indicating that as more months between product purchases go by, the less likely the individual is to spend, so marketing resources should not be wasted on individuals with a high rfm9.

Conclusion

The predictive modeling framework developed for B_TGT, INT_TGT, and CNT_TGT has successfully enabled a comprehensive understanding of customer behavior across multiple dimensions. By leveraging H2O AutoML and rigorous evaluation metrics, we've identified high-probability product adopters, estimated potential sales value with meaningful accuracy, and predicted the number of products customers are likely to adopt. Additionally, we leveraged more interpretable models alongside variable importance charts to generate actionable strategies for more effective targeting, personalized marketing strategies, and revenue forecasting. As a result, this modeling initiative not only enhances decision-making but also empowers the business to allocate resources more efficiently and drive measurable growth.

Appendix.a

Part 1: EDA

Table 1: Data Understanding

Data Type	Description	Columns
<i>Numeric</i>	Continuous variables or discrete numeric inputs.	18 Columns: int_tgt, cnt_tgt, rfm1-12, demog_age, demog_homeval, demog_inc, demog_pr
<i>Factor</i>	Categorical with fixed levels	6 Columns: b_tgt, cat_input1, cat_input2, demog_ho, demog_genf, demog_genm
<i>Identifier</i>	Unique identifiers for each row	1 Column: account
<i>Partition</i>	Subset of the dataset	1 Column: dataset

Table 2: Variable Descriptions

Variable	Type	Description
Target Variables		
<i>B_TGT</i>	Binary	Tried a New Product (Yes/No)
<i>INT_TGT</i>	Numeric	Total New Sales
<i>CNT_TGT</i>	Count	Count of New Products Purchased
Categorical Predictors		
<i>CAT_INPUT1</i>	Categorical	Account Activity
<i>CAT_INPUT2</i>	Categorical	Customer Value Level
RFM Interval Inputs		
<i>RFM1</i>	Numeric	Average Sales Past 3 Years
<i>RFM2</i>	Numeric	Average Sales Lifetime
<i>RFM3</i>	Numeric	Avg Sales Past 3 Years - Direct Promo Response
<i>RFM4</i>	Numeric	Last Product Purchase Amount
<i>RFM5</i>	Numeric	Count Purchased Past 3 Years
<i>RFM6</i>	Numeric	Count Purchased Lifetime
<i>RFM7</i>	Numeric	Count Purchased Past 3 Years - Direct Promo Response
<i>RFM8</i>	Numeric	Count Purchased Lifetime - Direct Promo Response
<i>RFM9</i>	Numeric	Months Since Last Purchase
<i>RFM10</i>	Numeric	Count Total Promos Past Year
<i>RFM11</i>	Numeric	Count Direct Promos Past Year
<i>RFM12</i>	Numeric	Customer Tenure
Demographic Inputs		
<i>DEMOG_AGE</i>	Numeric	Customer Age
<i>DEMOG_GENF</i>	Binary	Female (Yes/No)
<i>DEMOG_GENM</i>	Binary	Male (Yes/No)
<i>DEMOG_HO</i>	Binary	Homeowner (Yes/No)
<i>DEMOG_HOMEVAL</i>	Numeric	Home Value
<i>DEMOG_INC</i>	Numeric	Income
<i>DEMOG_PR</i>	Numeric	Geographical Retirement Percentage

Table 3: Missing Values

<i>Variable</i>	<i># Missing</i>	<i>% Missing</i>
<i>int_tgt</i>	848529	80.0
<i>demog_age</i>	266861	25.2
<i>rfm3</i>	225786	21.3
<i>cnt_tgt</i>	1	0

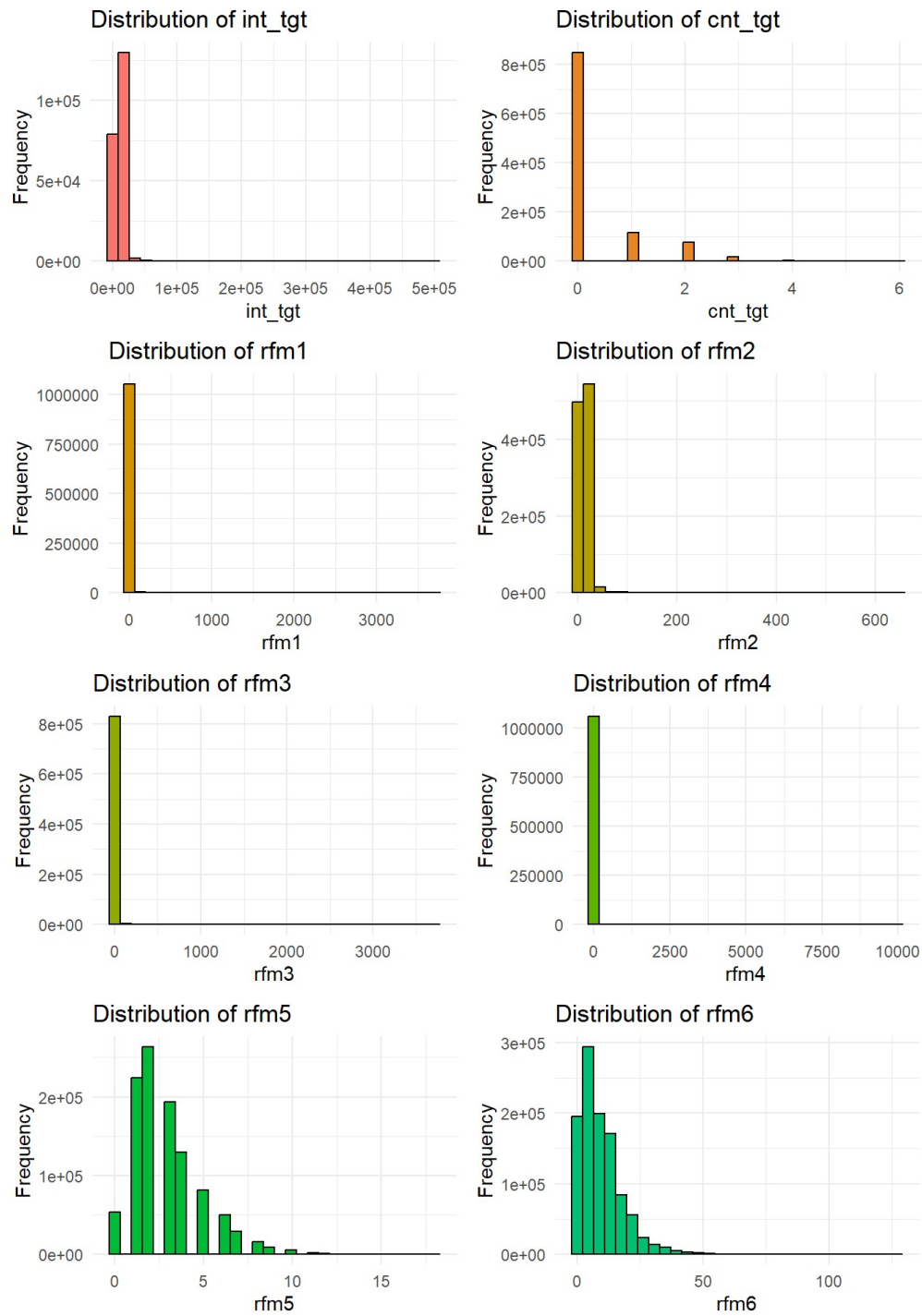
Table 4: Summary Statistics – Numerical

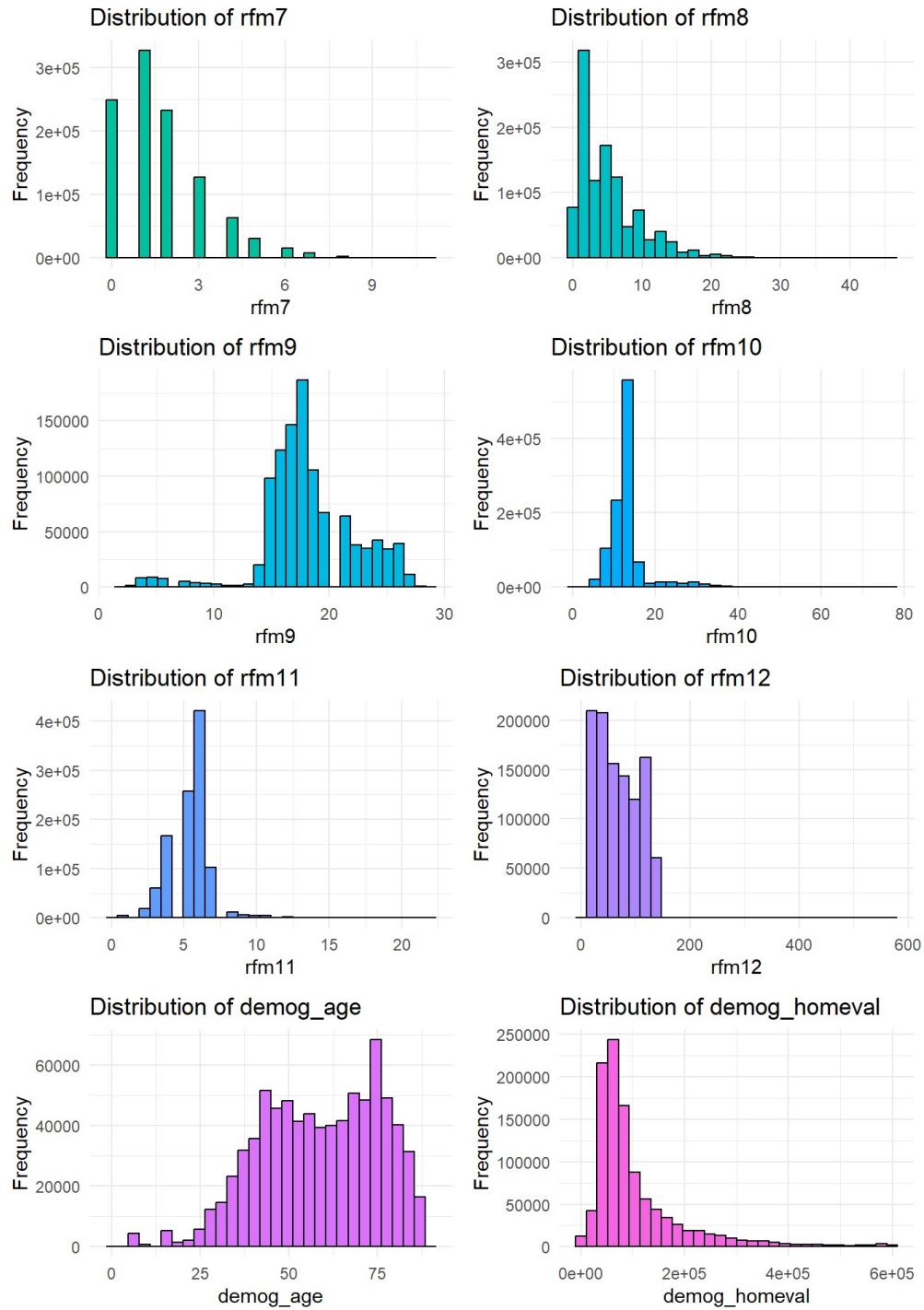
<i>Variable</i>	<i>Mean</i>	<i>Standard Deviation</i>	<i>Median</i>	<i>Min</i>	<i>Max</i>	<i>Skewness</i>
<i>int_tgt (>0)</i>	11235.87	8491.80	10000.00	0.00	500000.00	13.30
<i>cnt_tgt</i>	0.31	0.70	0.00	0.00	6.00	2.40
<i>rfm1</i>	16.09	19.30	15.00	0.00	3713.31	103.31
<i>rfm2</i>	13.35	9.47	11.67	1.58	650.00	11.12
<i>rfm3</i>	15.31	18.97	14.00	0.00	3713.31	114.92
<i>rfm4</i>	17.47	37.55	15.00	0.00	10000.00	207.18
<i>rfm5</i>	2.91	2.03	2.00	0.00	18.00	1.23
<i>rfm6</i>	9.54	8.47	7.00	0.00	127.00	1.91
<i>rfm7</i>	1.67	1.53	1.00	0.00	11.00	1.22
<i>rfm8</i>	5.03	4.51	4.00	0.00	46.00	1.42
<i>rfm9</i>	18.35	4.02	18.00	2.00	29.00	-0.60
<i>rfm10</i>	12.89	4.61	12.00	0.00	77.00	2.86
<i>rfm11</i>	5.36	1.36	6.00	0.00	22.00	0.32
<i>rfm12</i>	68.13	37.35	64.00	0.00	571.00	0.30
<i>demog_age</i>	58.72	16.85	60.00	-1.00	89.00	-0.36
<i>demog_homeval</i>	106103.55	93289.97	73880.00	0.00	600067.00	2.46
<i>demog_inc</i>	40368.69	28029.02	43174.00	0.00	200007.00	0.23
<i>demog_pr</i>	30.57	11.53	31.00	0.00	101.00	-0.15

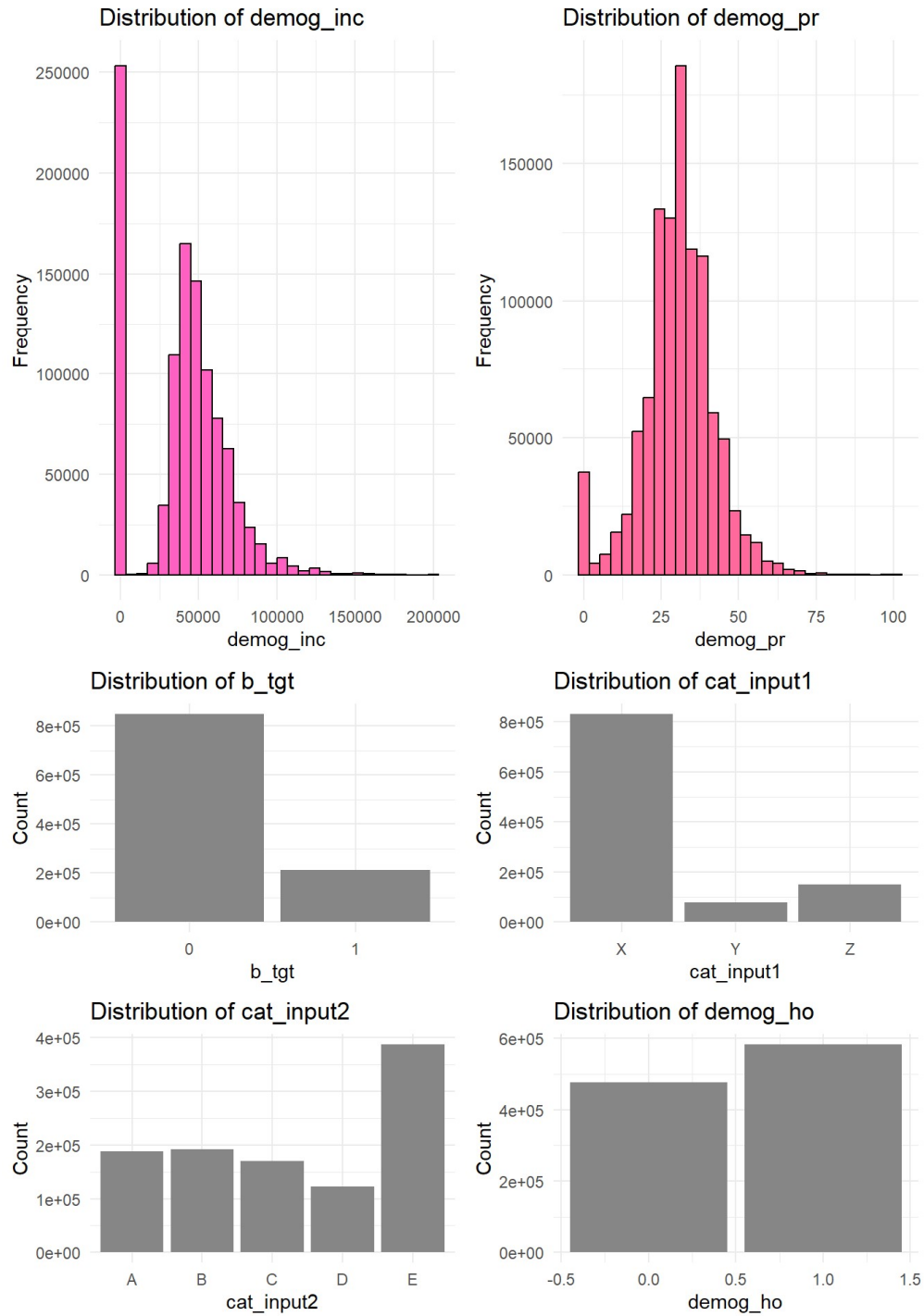
Table 5: Summary Statistics – Categorical/Binary

<i>Variable</i>	<i>Level</i>	<i>Count</i>	<i>Percent</i>
<i>b_tgt</i>	0	848529	80.05
	1	211509	19.95
<i>cat_input1</i>	X	831371	78.43
	Y	77847	7.34
	Z	150820	14.23
<i>cat_input2</i>	A	188398	17.77
	B	192382	18.15
	C	169550	15.99
	D	122282	11.54
	E	387426	36.55
<i>demog_ho</i>	0	476741	44.97
	1	583297	55.03
<i>demog_genf</i>	0	464288	43.80
	1	595750	56.20

Charts: Distributions and Counts







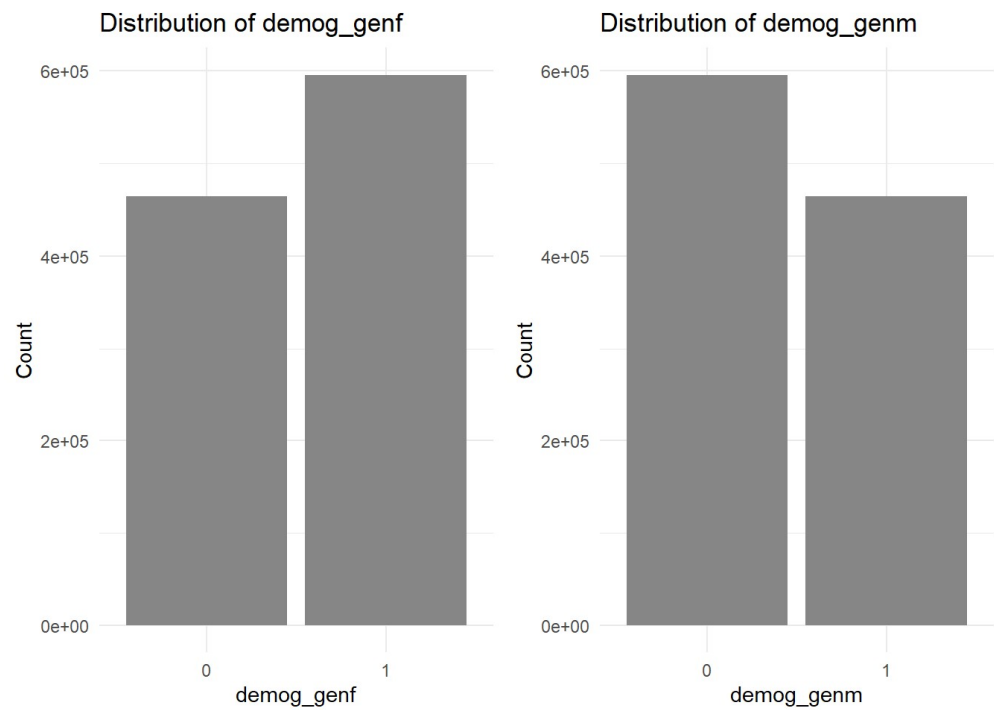


Chart 2: Correlation Heatmap

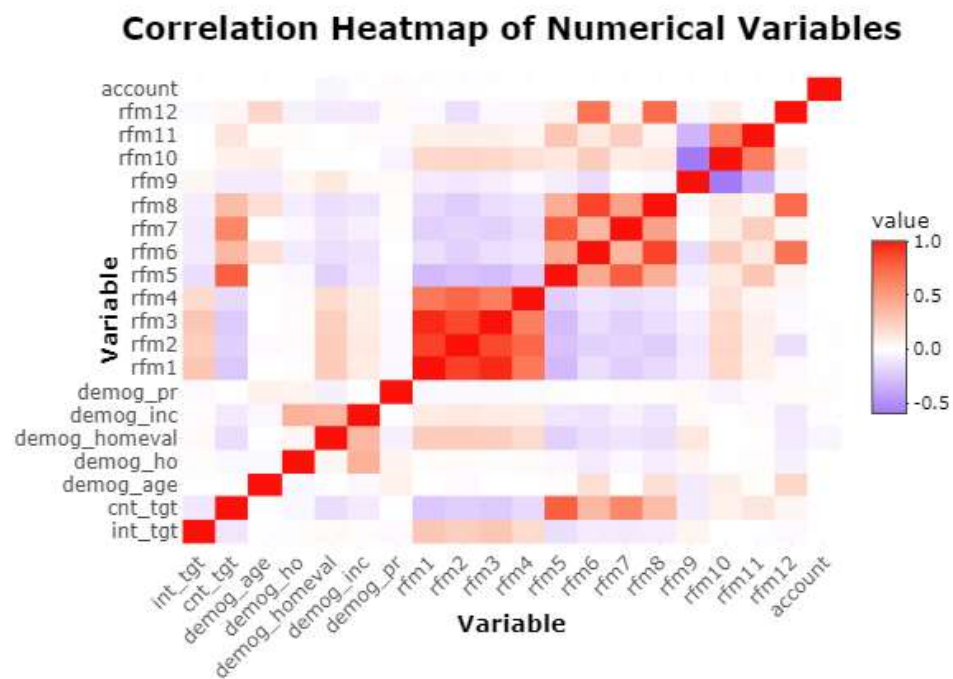


Table 6: Strong Correlations ($> |0.6|$)

Variable1	Variable2	Correlation
rfm3	rfm1	0.96
rfm2	rfm1	0.91
rfm8	rfm6	0.89
rfm3	rfm2	0.88
rfm7	rfm5	0.80
rfm5	cnt_tgt	0.79
rfm4	rfm2	0.76
rfm12	rfm8	0.74
rfm12	rfm6	0.71
rfm4	rfm1	0.69
rfm4	rfm3	0.66
rfm11	rfm10	0.66
rfm7	cnt_tgt	0.62

Table 7: Correlation Analysis

Correlation (Abs. Value)	Strength	Suggestion
0.0–0.3	Weak	Usually not worth flagging
0.3–0.5	Moderate	Context-dependent
0.5–0.7	Strong-ish	Often worth a closer look
0.7–0.9+	Very strong	Good candidates for multicollinearity check

Part 2: Modelling**Table 8:** Post Data Processing

Variable	mean	sd	median	min	max	skewness
int_tgt	2583.73	5931.40	0.00	0.00	200000.0	4.77
cnt_tgt	0.37	0.76	0.00	0.00	5.0	2.09
rfm1	15.44	15.09	13.98	1.15	2507.5	84.12
rfm2	12.80	8.93	11.10	1.58	436.0	9.92
rfm3	15.22	11.10	14.00	0.00	600.0	9.85
rfm4	16.70	44.24	15.00	0.00	10000.0	205.74
rfm5	3.29	2.05	3.00	0.00	18.0	1.10
rfm6	10.34	8.78	8.00	0.00	127.0	1.77
rfm7	2.07	1.46	2.00	0.00	11.0	1.24
rfm8	5.67	4.61	4.00	0.00	36.0	1.31
rfm9	18.18	3.87	18.00	2.00	28.0	-0.73
rfm10	13.07	4.51	13.00	0.00	60.0	2.91
rfm11	5.50	1.29	6.00	0.00	17.0	0.47
rfm12	69.32	37.36	65.00	13.00	571.0	0.30
demog_age	59.11	16.43	60.00	16.00	89.0	-0.24
demog_homeval	106619.64	92456.21	74908.00	0.00	600067.0	2.50
demog_inc	46562.05	25211.91	46450.00	0.00	200007.0	0.24
demog_pr	31.16	11.24	31.00	0.00	100.0	-0.09

Table 9: Logged Variable Statistics

<i>Variable</i>	<i>mean</i>	<i>sd</i>	<i>median</i>	<i>min</i>	<i>max</i>	<i>skewness</i>
<i>log_int_tgt</i>	2.09	3.84	0.00	0.00	12.21	1.31
<i>log_rfm1</i>	2.67	0.48	2.71	0.77	7.83	0.25
<i>log_rfm2</i>	2.51	0.47	2.49	0.95	6.08	0.36
<i>log_rfm3</i>	2.65	0.50	2.71	0.00	6.40	0.16
<i>log_rfm4</i>	2.71	0.54	2.77	0.00	9.21	-0.30
<i>log_rfm10</i>	2.61	0.26	2.64	0.00	4.11	0.97

Table 10: Int_tgt Validation

<i>Model Rank</i>	<i>Type</i>	<i>RMSE</i>	<i>MSE</i>	<i>Unlogged_RMSE</i>	<i>r2</i>
<i>Model 1</i>	drf	1.4412	2.0770	2.64	0.8599
<i>Model 2</i>	gbm	1.7095	2.9224	4.19	0.8029
<i>Model 3</i>	deeplearning	3.1637	10.0093	149.10	0.3251
<i>Model 4</i>	glm	3.3184	11.0119	246.15	0.2575

Table 11: Best Model

<i>Model Rank</i>	<i>Type</i>	<i>RMSE</i>	<i>MSE</i>	<i>Unlogged_RMSE</i>	<i>r2</i>
<i>Final Model (Model 1)</i>	drf	1.4457	2.0902	2.66	0.8585

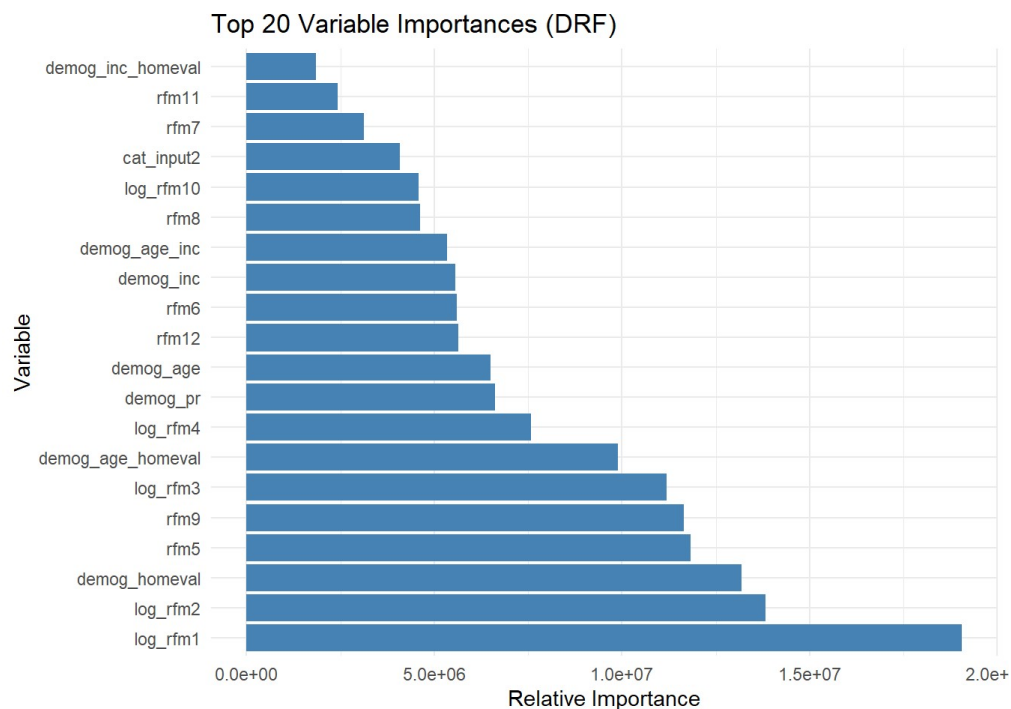
Chart 3: Variable Importance for Int_tgt

Chart 4: Predicted vs Actual Plot
 Predicted vs Actual for Int Target

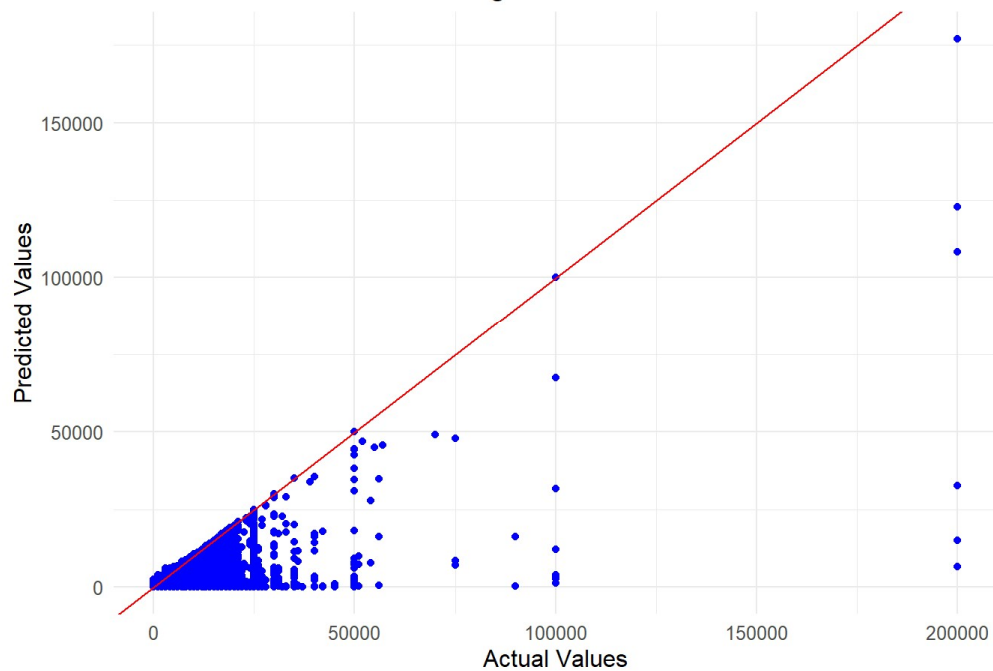


Table 12: int_tgt GLM Coeffs

Predictor	Coefficient	Odds Ratio (%)
Intercept	9.1656809	956223.08
cat_input2.A	0.1102875	11.66
cat_input2.B	0.0886604	9.27
cat_input2.C	-0.0050504	-0.50
cat_input2.D	-0.0575689	-5.59
cat_input2.E	-0.1377800	-12.87
cat_input1.X	0.0404378	4.13
cat_input1.Y	-0.0142278	-1.41
cat_input1.Z	-0.0243132	-2.40
demog_genf.0	-0.0134046	-1.33
demog_genf.1	0.0153435	1.55
demog_age	-0.0042269	-0.42
demog_ho	-0.0318255	-3.13
demog_homeval	0.0000085	0.00
demog_inc	0.0000022	0.00
demog_pr	-0.0011127	-0.11
rfm5	0.4166238	51.68
rfm6	-0.0062728	-0.63
rfm7	-0.1152239	-10.88
rfm8	0.0249737	2.53
rfm9	-0.1990148	-18.05
rfm11	0.0126836	1.28

rfm12	0.0026433	0.26
log_rfm1	-0.9859614	-62.69
log_rfm2	-1.1087890	-67.00
log_rfm3	0.1029872	10.85
log_rfm4	-0.0548908	-5.34
log_rfm10	-0.1889794	-17.22
demog_inc_homeval	0.0000000	0.00
demog_age_homeval	0.0000000	0.00
demog_age_inc	0.0000000	0.00

Table 13: b_tgt Validation

Model Rank	Type	AUC
<i>Model 1</i>	gbm	0.9970
<i>Model 2</i>	drf	0.9961
<i>Model 3</i>	deeplearning	0.8657
<i>Model 4</i>	glm	0.8521

Table 14: Best Model b_tgt

Model Rank	Type	AUC
<i>Best Model (Model 1)</i>	gbm	0.9965

Chart 5: Variable Importance for b_tgt

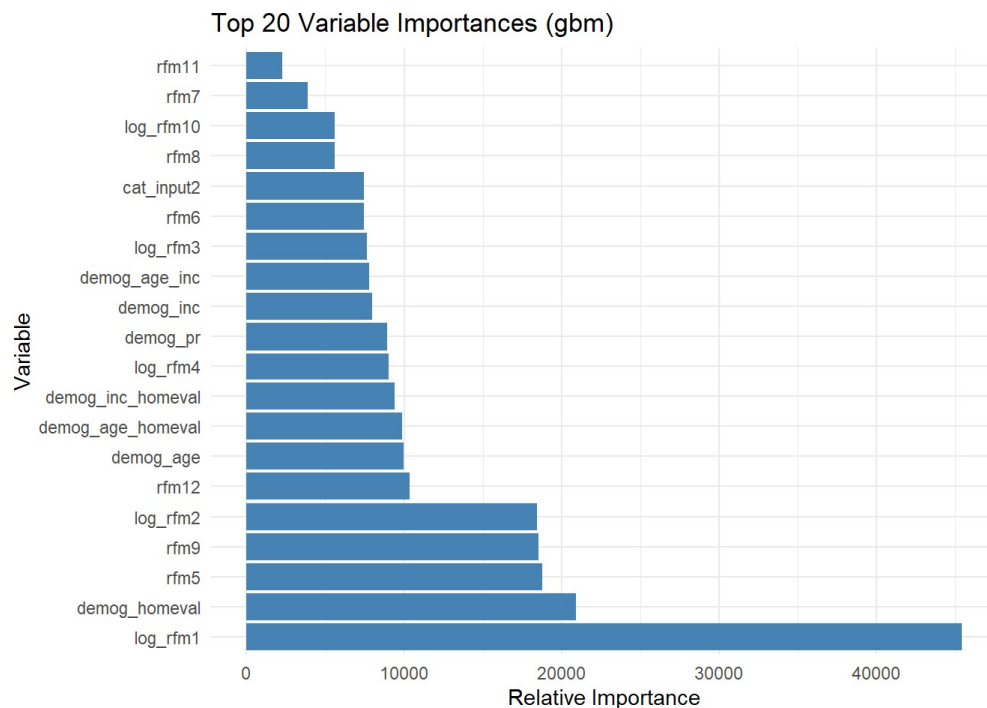


Chart 6: Visualization of the AUC

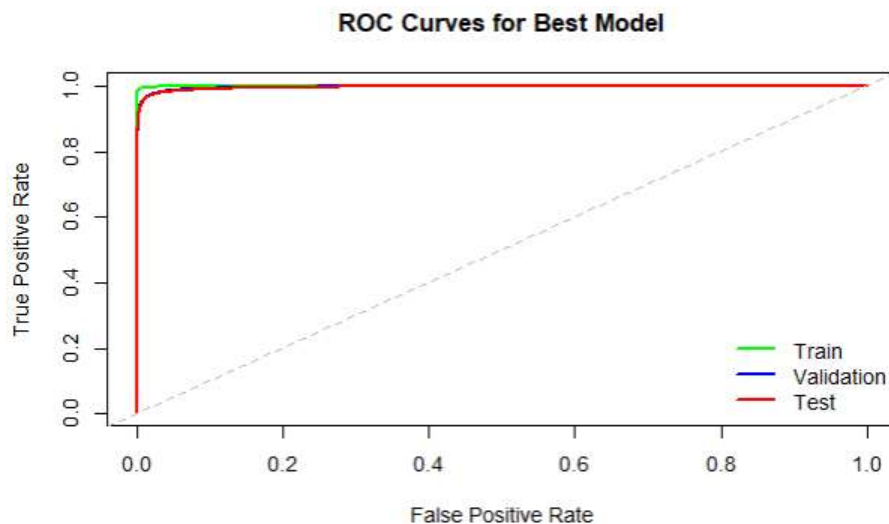


Table 15: GLM Coefficients for b_tgt

Predictor	Coefficient	Odds Ratio (%)
Intercept	5.0998005	16298.92
cat_input2.A	0.1119309	11.84
cat_input2.B	0.0942768	9.89
cat_input2.C	-0.0037163	-0.37
cat_input2.D	-0.0437802	-4.28
cat_input2.E	-0.1597183	-14.76
cat_input1.X	0.1709740	18.65
cat_input1.Y	0.1377629	14.77
cat_input1.Z	-0.3239737	-27.67
demog_genf.0	-0.0148939	-1.48
demog_genf.1	0.0032531	0.33
demog_age	-0.0015614	-0.16
demog_ho	-0.0298982	-2.95
demog_homeval	0.0000078	0.00
demog_inc	0.0000041	0.00
demog_pr	0.0006671	0.07
rfm5	0.2830672	32.72
rfm6	-0.0106399	-1.06
rfm7	-0.0864276	-8.28
rfm8	0.0154354	1.56
rfm9	-0.1787102	-16.37
rfm11	0.0672027	6.95
rfm12	0.0027677	0.28
log_rfm1	-0.7637720	-53.41
log_rfm2	-1.3508529	-74.10

log_rfm3	0.0610994	6.30
log_rfm4	0.0424615	4.34
log_rfm10	-0.2590137	-22.82
demog_inc_homeval	0.0000000	0.00
demog_age_homeval	0.0000000	0.00
demog_age_inc	0.0000000	0.00

Table 16: Best Model Confusion Matrix

	0	1	Error	Rate
0	93280	875	0.0092932	=875/94155
1	1213	27490	0.0422604	=1213/28703
Totals	94493	28365	0.0169952	=2088/122858

Table 17: cnt_tgt Validation

Model	Type	RMSE	MSE
Model 1	drf	0.2574	0.0663
Model 2	gbm	0.2754	0.0758
Model 3	deeplearning	0.5745	0.3301
Model 4	glm	0.6165	0.3800

Table 18: Best Model

Model	Type	RMSE	MSE
Best Model (Test Set)	drf	0.2562	0.0656

Chart 7: Variable Importance cnt_tgt

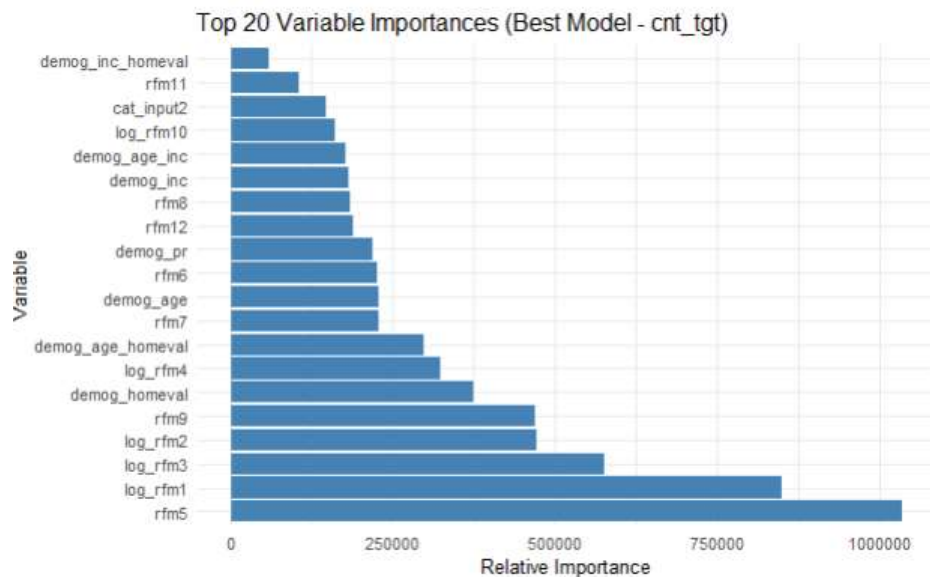
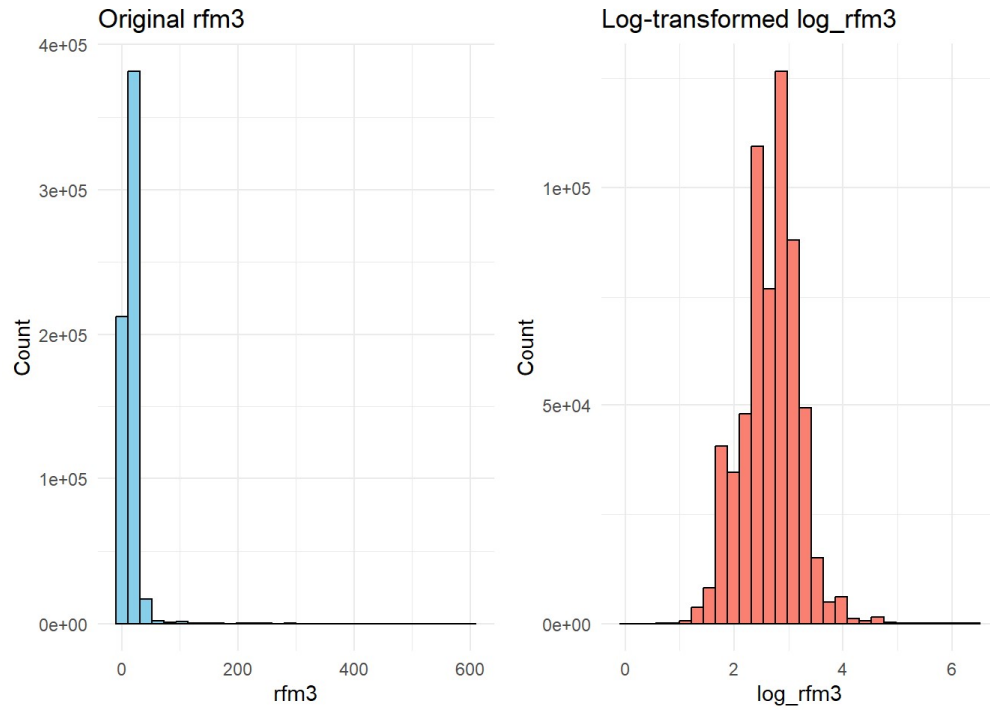


Chart 8: Actual vs Predicted for cnt_tgt**Table 19:**

Predictor	Coefficient	Odds Ratio (%)
Intercept	1.5087728	352.12
cat_input2.A	0.0181896	1.84
cat_input2.B	0.0232440	2.35
cat_input2.C	-0.0044892	-0.45
cat_input2.D	-0.0100478	-1.00
cat_input2.E	-0.0272705	-2.69
cat_input1.X	-0.0171031	-1.70
cat_input1.Y	-0.0281818	-2.78
cat_input1.Z	0.0487102	4.99
demog_genf.0	0.0003223	0.03
demog_genf.1	0.0023244	0.23
demog_age	-0.0009596	-0.10
demog_ho	-0.0055315	-0.55
demog_homeval	0.0000013	0.00
demog_inc	0.0000004	0.00
demog_pr	-0.0003986	-0.04
rfm5	0.1519339	16.41
rfm6	0.0021239	0.21
rfm7	-0.0164522	-1.63
rfm8	0.0029292	0.29
rfm9	-0.0364782	-3.58
rfm11	-0.0255904	-2.53
rfm12	-0.0001056	-0.01
log_rfm1	-0.1606913	-14.84
log_rfm2	-0.1890883	-17.23

log_rfm3	0.0418248	4.27
log_rfm4	-0.0147741	-1.47
log_rfm10	-0.0331659	-3.26
demog_inc_homeval	0.0000000	0.00
demog_age_homeval	0.0000000	0.00
demog_age_inc	0.0000000	0.00

Chart 9: Transformation Visualization

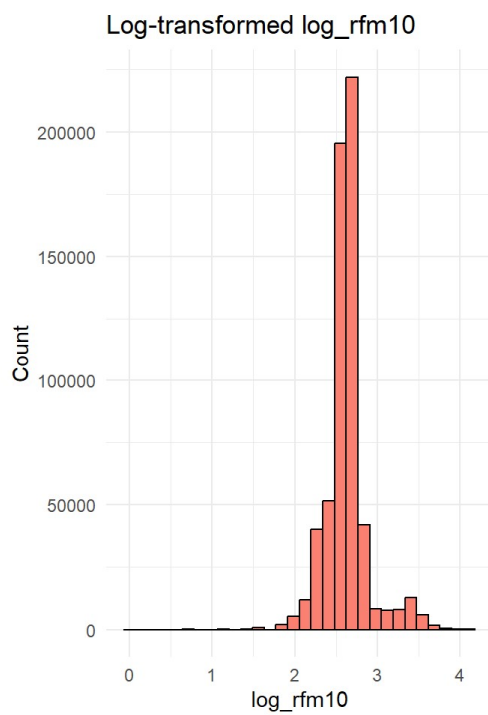
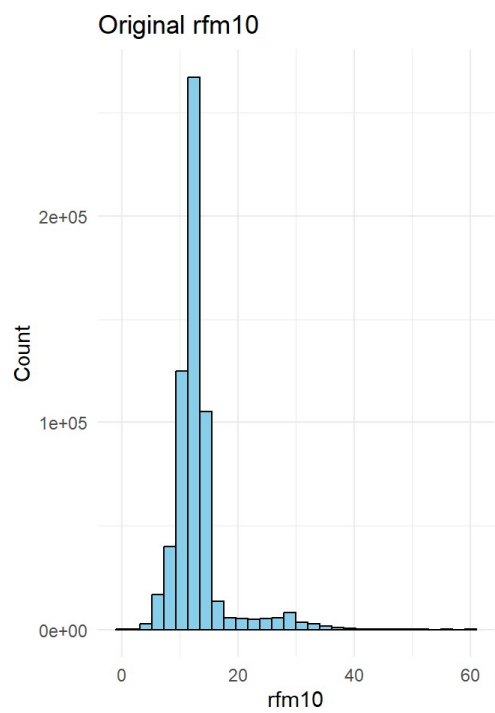
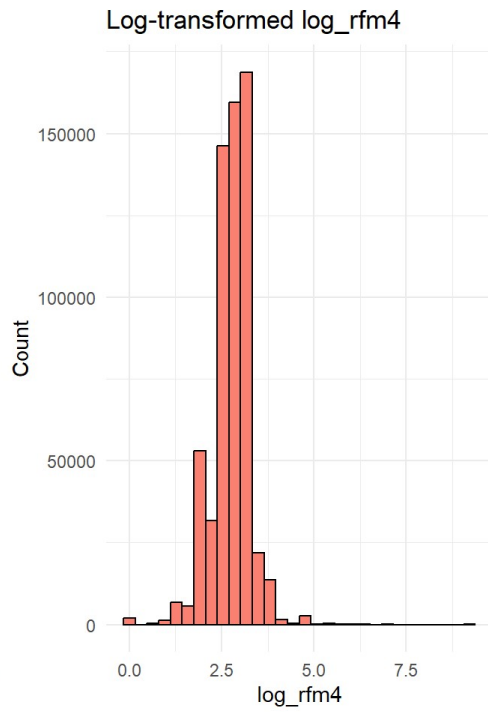
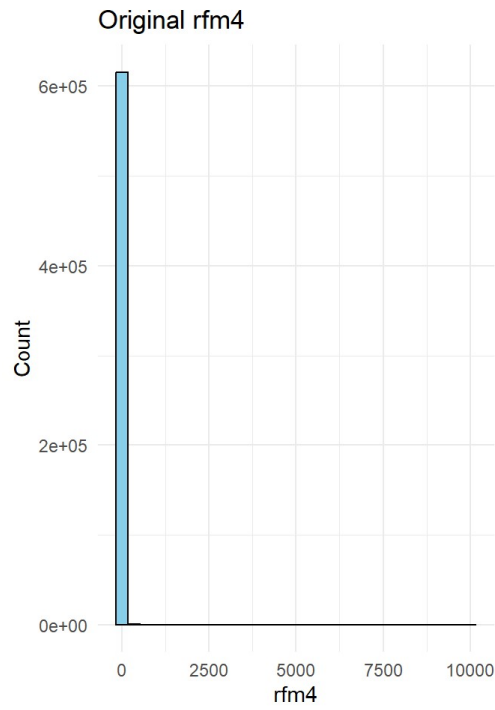


Table 20: Categorical/Binary Counts after Missing Removal

<i>Variable</i>	<i>Level</i>	<i>Count</i>	<i>Percent</i>
<i>b_tgt</i>	0	471669	76.61
<i>b_tgt</i>	1	144045	23.39
<i>cat_input1</i>	X	498539	80.97
<i>cat_input1</i>	Y	37624	6.11
<i>cat_input1</i>	Z	79551	12.92
<i>cat_input2</i>	A	110689	17.98
<i>cat_input2</i>	B	118279	19.21
<i>cat_input2</i>	C	101607	16.50
<i>cat_input2</i>	D	71510	11.61
<i>cat_input2</i>	E	213629	34.70
<i>demog_ho</i>	0	206092	33.47
<i>demog_ho</i>	1	409622	66.53
<i>demog_genf</i>	0	272906	44.32
<i>demog_genf</i>	1	342808	55.68

Appendix.b.1

```

---
title: "Final Project"
author: "Shaun Levenson"
date: "`r Sys.Date()`"
output:
  html_document:
    toc: true
    toc_float: true
    self_contained: true
    code_folding: hide
---

```{r packages, echo=TRUE}
Install the required packages only if missing and suppress redundant output
packages <- c("tidyverse", "caret", "knitr", "kableExtra", "skimr",
 "ggplot2", "dplyr", "ISLR", "tree", "rpart", "rpart.plot", "randomForest", "gb
 m", "doParallel", "naniar", "patchwork", "MASS", "scales", "plotly", "reshape2
 ", "e1071", "ggcorrplot", "gt", "nortest", "gridExtra")
new_packages <- setdiff(packages, rownames(installed.packages()))
if (length(new_packages) > 0) install.packages(new_packages, quietly = TRUE)

Load the required packages quietly
invisible(lapply(packages, function(pkg)
 suppressPackageStartupMessages(library(pkg, character.only = TRUE))))
...

1. Data Loading and Inspection

```{r load data, echo=TRUE}
#Import the dataset. Change the file path to where you saved your bank.txt
file
bank <- read_csv("F:/ECON/562_Analytics_2/Final Project/bank.txt",
  col_types = cols(b_tgt = col_character(),
    int_tgt = col_number(), cnt_tgt =
    col_double(),
    demog_homeval = col_number(), demog_inc =
    col_number(),
    rfm1 = col_number(), rfm2 = col_number(),
    rfm3 = col_number(), rfm4 = col_number(),
    demog_genf = col_character(), demog_genm =
    col_character(),
    dataset = col_character()))

# Display the first 10 rows of the dataset
bank %>%
  slice(1:10) %>%
  kable("html", caption = "First 10 Rows of the Bank Dataset") %>%
  kable_styling(bootstrap_options = c("striped", "hover", "condensed",
    "responsive"),
    full_width = FALSE,
    position = "center")

str(bank)
summary(bank)

```

```

...

## Data Understanding

|Data Type | Description | Columns |
|-----|-----|-----|
|**Numeric** | Continuous variables or discrete numeric inputs. | 18 Columns:
    `int_tgt`, `cnt_tgt`, `rfm1-12`, `demog_age`, `demog_homeval`,
    `demog_inc`, `demog_pr`|
|**Factor** | Categorical with fixed levels | 6 Columns: `b_tgt`,
    `cat_input1`, `cat_input2`, `demog_ho`, `demog_genf`, `demog_genm` |
|**Identifier** | Unique identifiers for each row | 1 Column: `account` |
|**Partition** | Subset of the dataset | 1 Column: `dataset` |

**Variable Descriptions**

| Variable | Type | Description |
|-----|-----|-----|
|-----|-----|-----|
| **Target Variables** |||
| `B_TGT` | Binary | Tried a New Product (Yes/No – may be recoded as 1/0) |
| `INT_TGT` | Numeric | Total New Sales |
| `CNT_TGT` | Count | Count of New Products Purchased |
| **Categorical Predictors** |||
| `CAT_INPUT1` | Categorical | Account Activity |
| `CAT_INPUT2` | Categorical | Customer Value Level |
| **RFM Interval Inputs** |||
| `RFM1` | Numeric | Average Sales Past 3 Years |
| `RFM2` | Numeric | Average Sales Lifetime |
| `RFM3` | Numeric | Avg Sales Past 3 Years - Direct Promo Response |
| `RFM4` | Numeric | Last Product Purchase Amount |
| `RFM5` | Numeric | Count Purchased Past 3 Years |
| `RFM6` | Numeric | Count Purchased Lifetime |
| `RFM7` | Numeric | Count Purchased Past 3 Years - Direct Promo Response |
| `RFM8` | Numeric | Count Purchased Lifetime - Direct Promo Response |
| `RFM9` | Numeric | Months Since Last Purchase |
| `RFM10` | Numeric | Count Total Promos Past Year |
| `RFM11` | Numeric | Count Direct Promos Past Year |
| `RFM12` | Numeric | Customer Tenure |
| **Demographic Inputs** |||
| `DEMOG_AGE` | Numeric | Customer Age |
| `DEMOG_GENF` | Binary | Female (Yes/No) |
| `DEMOG_GENM` | Binary | Male (Yes/No) |
| `DEMOG_HO` | Binary | Homeowner (Yes/No) |
| `DEMOG_HOMEVAL` | Numeric | Home Value |
| `DEMOG_INC` | Numeric | Income |
| `DEMOG_PR` | Numeric | Geographical Retirement Percentage |

**General Structure**

- The dataset contains **1,060,038 rows** and **26 columns.**

- The dataset is divided into three subsets: training (1), validation (2),
  and testing (3).

- 3 target variables

```

- **b_tgt**: Binary target variable indicating whether the customer tried a new product (yes/no).
- **int_tgt**: Numeric target variable indicating the total number of new sales.
- **cnt_tgt**: Count target variable indicating the total number of new products and services purchased by customers.

Potential Issues

- Many missing values
- Redundant Gender columns: do not need both male and female. Will only keep the female column.
- Zero values in `demog_homeval`, `demog_inc`, and `demog_pr` columns. These values are suspicious and may be treated as missing values.

2. Missing Values

```
```{r missing values, echo=TRUE}
Missing Value Table
bank %>%
 miss_var_summary() %>%
 mutate(pct_miss = round(pct_miss, 2)) %>%
 kable("html", caption = "Missing Data Summary for 'bank' Dataset") %>%
 kable_styling(bootstrap_options = c("striped", "hover", "condensed",
 "responsive"),
 full_width = F,
 position = "center")
```
```

Important Note

- The dataset contains a significant amount of missing values, particularly in the `int_tgt` variable, which has **80%** missing values. This is not a major concern due to the nature of the modelling focus. The `int_tgt` variable is set to N/A for any `b_tgt` that is "no" and since we will be modelling for how much a customer will spend given they have purchased a new product, we can effectively ignore all missing values for this variable.
- The other two highly missing variables are `demog_age` and `rfm3`, which have **25.2%** and **21.3%** missing values, respectively. These variables will be imputed using the median or mean values depending on the normality due to the importance of these variables.

3. Univariate Analysis

```
```{r summary statistics, echo=TRUE}
Numerical variable names
numerical_vars <- c("int_tgt", "cnt_tgt", "rfm1", "rfm2", "rfm3", "rfm4",
 "rfm5", "rfm6", "rfm7", "rfm8", "rfm9", "rfm10",
 "rfm11", "rfm12", "demog_age", "demog_homeval",
 "demog_inc", "demog_pr")
```

```
Function to compute summary statistics
compute_summary_statistics <- function(bank, numerical_vars) {
```

```

summary_stats <- bank %>%
 dplyr::select(dplyr::all_of(numerical_vars)) %>%
 dplyr::summarise(dplyr::across(
 everything(),
 list(
 mean = ~mean(., na.rm = TRUE),
 sd = ~sd(., na.rm = TRUE),
 median = ~median(., na.rm = TRUE),
 min = ~min(., na.rm = TRUE),
 max = ~max(., na.rm = TRUE),
 skewness = ~e1071::skewness(., na.rm = TRUE)
),
 .names = "{col}_{fn}"
)) %>%
 tidyr::pivot_longer(cols = everything(),
 names_to = c("Variable", "Statistic"),
 names_pattern = "(.*)_(.*)",
 values_to = "Value") %>%
 tidyr::pivot_wider(names_from = Statistic, values_from = Value) %>%
 dplyr::mutate(dplyr::across(where(is.numeric), ~round(.x, 2)))

return(summary_stats)
}

Run the function
summary_statistics <- compute_summary_statistics(bank, numerical_vars)

Display nicely formatted HTML table
summary_statistics %>%
 kable("html", caption = "Summary Statistics for Selected Numerical
 Variables") %>%
 kable_styling(full_width = TRUE, bootstrap_options = c("striped", "hover"))
 %>%
 scroll_box(width = "100%", height = "500px")
```



```

```{r categorical_variables_stats, echo=TRUE}
# List of categorical variable names
categorical_vars <- c("b_tgt", "cat_input1", "cat_input2", "demog_ho",
  "demog_genf")

# Function to compute frequency tables
compute_frequency_tables <- function(bank, categorical_vars) {
  freq_tables <- lapply(categorical_vars, function(var) {
    bank %>%
      dplyr::select(dplyr::all_of(var)) %>%
      dplyr::filter(!is.na(.data[[var]])) %>%
      dplyr::group_by(.data[[var]]) %>%
      dplyr::summarise(Count = n(), .groups = "drop") %>%
      dplyr::mutate(
        Level = as.character(.data[[var]]), # Force Level to be character
        Percent = round(100 * Count / sum(Count), 2),
        Variable = var
      ) %>%
      dplyr::select(Variable, Level, Count, Percent)
  })
}

```


```

```

Bind all tables into one
freq_tables_df <- dplyr::bind_rows(freq_tables)

return(freq_tables_df)
}

Run the function
frequency_tables <- compute_frequency_tables(bank, categorical_vars)

Display nicely formatted HTML table
frequency_tables %>%
 kable("html", caption = "Frequency Tables for Selected Categorical
 Variables") %>%
 kable_styling(full_width = TRUE, bootstrap_options = c("striped", "hover"))
 %>%
 scroll_box(width = "100%", height = "500px")
```

```{r univariate analysis visualization, echo=TRUE, warning=FALSE,
 message=FALSE}
Define all numerical variables
numerical_vars <- c("int_tgt", "cnt_tgt", paste0("rfm", 1:12),
 "demog_age", "demog_homeval", "demog_inc", "demog_pr")

Generate a vector of distinct colors
colors <- hue_pal()(length(numerical_vars))

Create a list to store plots
num_plots <- list()

for (i in seq_along(numerical_vars)) {
 var <- numerical_vars[i]
 color <- colors[i]

 plot_data <- bank %>% filter(!is.na(.data[[var]]))

 p <- ggplot(plot_data, aes(x = .data[[var]])) +
 geom_histogram(bins = 30, fill = color, color = "black") +
 labs(title = paste("Distribution of", var), x = var, y = "Frequency") +
 theme_minimal()

 num_plots[[i]] <- p
}

Display in 2x2 grids
num_chunks <- ceiling(length(num_plots) / 4)
for (i in 1:num_chunks) {
 idx <- ((i - 1) * 4 + 1):min(i * 4, length(num_plots))
 grid.arrange(grobs = num_plots[idx], ncol = 2)
}

Now do the same for categorical variables
categorical_vars <- c("b_tgt", "cat_input1", "cat_input2", "demog_ho",
 "demog_genf", "demog_genm")

```

```

cat_plots <- list()

for (var in categorical_vars) {
 p <- ggplot(bank, aes(x = .data[[var]])) +
 geom_bar(fill = "#868686FF") +
 labs(title = paste("Distribution of", var), x = var, y = "Count") +
 theme_minimal()

 cat_plots[[length(cat_plots) + 1]] <- p
}

Display in 2x2 grids
cat_chunks <- ceiling(length(cat_plots) / 4)
for (i in 1:cat_chunks) {
 idx <- ((i - 1) * 4 + 1):min(i * 4, length(cat_plots))
 grid.arrange(grobs = cat_plots[idx], ncol = 2)
}
...

```

## ## Analysis

The notable observations from these distributions for consideration:

### \*\*`b\_tgt`\*\*:

- This variable exhibits a significant imbalance, with a large number of "no" responses compared to "yes". This is common in marketing datasets where the target event (e.g., trying a new product) is rare.
- Class imbalance could cause models to favor the majority class, reducing predictive accuracy for the minority class. Addressing this is vital for effective model performance, especially in classification tasks.

### \*\*`int\_tgt`\*\*:

- This variable shows high skewness, indicating a right-skewed distribution. This suggests that most customers have low total new sales, with a few customers having very high sales. This chart may also indicate the presence of outliers by the long right tail. Previously, the summary statistic showed a max value of 500000, which is likely a significant outlier. This will be properly addressed in the data cleaning section to return to a normal distribution as many regression models assume normality.
- This histogram does not include the missing values, which are set to NA. This is important to note as the missing values are not included in the analysis.

### \*\*`cnt\_tgt`\*\*:

- This variable also shows a right-skewed distribution, similar to `int\_tgt`. This is due to most customers having 0 new products purchased, with a few customers having purchased multiple products. However, the summary statistics show that the skew of this variable is much lower than `int\_tgt`, indicating that the distribution is more normal, yet still may benefit from transformations.

## # 4. Bivariate Analysis



```

```{r correlation matrix, echo=TRUE}
# Compute the correlation matrix for numeric columns in the bank dataset
correlation_matrix <- cor(bank[, sapply(bank, is.numeric)], use =
  "complete.obs")

# Melt the correlation matrix to long format
melted_correlation <- melt(correlation_matrix)

# Truncate the correlation values to the nearest hundredth
melted_correlation$value <- trunc(melted_correlation$value * 100) / 100

# Create a heatmap using ggplot2
heatmap_plot <- ggplot(data = melted_correlation, aes(Var1, Var2, fill =
  value)) +
  geom_tile(color = "white") +
  scale_fill_gradient2(low = "blue", mid = "white", high = "red", midpoint =
    0) +
  labs(title = "Correlation Heatmap of Numerical Variables",
    x = "Variable",
    y = "Variable") +
  theme_minimal() +
  theme(
    plot.title = element_text(hjust = 0.5, size = 16, face = "bold"),
    axis.title.x = element_text(size = 12, face = "bold"),
    axis.title.y = element_text(size = 12, face = "bold"),
    axis.text.x = element_text(angle = 45, hjust = 1, vjust = 1, size = 10),
    axis.text.y = element_text(size = 10)
  )

# Convert to interactive plot
interactive_heatmap <- ggplotly(heatmap_plot, tooltip = c("Var1", "Var2",
  "value"))

# Show the heatmap
interactive_heatmap
```

```

```

```{r correlations, echo=TRUE}
# Compute correlation matrix
cor_matrix <- cor(bank %>% select(where(is.numeric)), use = "complete.obs")

# Melt to long format
cor_long <- melt(cor_matrix)

# Remove self-correlations and duplicate pairs
cor_filtered <- cor_long %>%
  filter(Var1 != Var2) %>%
  rowwise() %>%
  mutate(pair = paste(sort(c(Var1, Var2)), collapse = "_")) %>%
  distinct(pair, .keep_all = TRUE) %>%
  filter(abs(value) > 0.6) %>%
  arrange(desc(abs(value)))

# Create a nice table
cor_table <- cor_filtered %>%

```

```

select(Variable1 = Var1, Variable2 = Var2, Correlation = value) %>%
mutate(Correlation = round(Correlation, 2)) %>%
kable("html", caption = "Strong Correlations (>|0.6|)", escape = FALSE) %>%
kable_styling(bootstrap_options = c("striped", "hover", "condensed"),
  full_width = FALSE)

# Display
cor_table
```

Analysis

Correlation (Abs. Value)	Strength	Suggestion
0.0-0.3	Weak	Usually not worth flagging
0.3-0.5	Moderate	Context-dependent
0.5-0.7	Strong-ish	Often worth a closer look
0.7-0.9+	Very strong	Good candidates for multicollinearity check

- There is extremely high correlation across the RFM variables, especially `rfm1`, `rfm2`, and `rfm3`.

- `rfm1` (Avg Sales past 3 yrs) is highly correlated with:
 - `rfm2` (Avg Sales Lifetime): 0.91
 - `rfm3` (Avg Sales Past 3 yrs Dir Promo Resp): 0.96

This is expected as these variables are all related to average sales over different time periods.

- `rfm6` (Count Purchased Lifetime) and `rfm8` (Count Purchased Lifetime Dir Promo Resp): 0.89

- `rfm5` (Count Purchased Past 3 Yrs) and `cnt_tgt` (Target variable): 0.79

Modelling Approach

1. Multicollinearity Warning

- Extremely high correlations (e.g., > 0.9), like between `rfm1` and `rfm3`, can cause multicollinearity, which may:
 - Distort the interpretation of coefficients in linear models
 - Inflate standard errors

- **Solution**: Consider removing one of each highly correlated pair (e.g., keep either `rfm1` or `rfm3`, not both), or use dimension reduction techniques such as:
 - Principal Component Analysis (PCA)
 - Regularization (e.g., Ridge, Lasso)

2. Feature Selection

- Strong correlation between some predictors (e.g., `rfm5`) and the target variable (`cnt_tgt`) suggests these may be strong predictors:

```

- These should be prioritized in model training or feature selection algorithms.

### ### 3. Model Choice Consideration

- Linear models are sensitive to multicollinearity, but tree-based models (like Random Forest or Gradient Boosting) are more robust in this context.
- For models where interpretability matters, handling multicollinearity is crucial.

### # 5. Data Quality and Outliers

```

```{r log transformation, echo=TRUE}
# Log transformation for skewed variables
skewed_vars <- c("int_tgt", "cnt_tgt", "rfm1", "rfm2", "rfm3", "rfm4",
  "rfm10", "demog_homeval")

bank[paste("log_", skewed_vars, sep = "_")] <- lapply(bank[skewed_vars],
  function(x) {
    if (any(x <= 0, na.rm = TRUE)) {
      x[x <= 0] <- NA
    }
    log(x)
  })

# Check the distribution of the transformed variables
skewed_vars_transformed <- bank %>%
  select(starts_with("log_")) %>%
  gather(key = "Variable", value = "Value") %>%
  filter(!is.na(Value))

ggplot(skewed_vars_transformed, aes(x = Value)) +
  geom_histogram(bins = 30, fill = "#8686FF", color = "black") +
  facet_wrap(~ Variable, scales = "free") +
  labs(title = "Distribution of Log-Transformed Variables", x = "Log Value",
    y = "Count") +
  theme_minimal()

...

```{r outlier checks, echo=TRUE, warning=FALSE}
Function to generate boxplot and return both plot and outlier count
outlier_check_plot <- function(data, var) {
 # Generate boxplot
 p <- ggplot(data, aes(y = .data[[var]])) +
 geom_boxplot(fill = "#00AFBB", color = "black", outlier.color = "red",
 outlier.shape = 16) +
 labs(title = paste("Boxplot of", var), y = var) +
 theme_minimal()

 # Identify outliers using 1.5*IQR rule
 qnt <- quantile(data[[var]], probs = c(0.25, 0.75), na.rm = TRUE)
 iqr <- qnt[2] - qnt[1]
 lower <- qnt[1] - 1.5 * iqr
 upper <- qnt[2] + 1.5 * iqr

```

```

 outliers <- which(data[[var]] < lower | data[[var]] > upper)

 return(list(plot = p, count = length(outliers)))
}

Define variables to check
variables_to_check <- c(
 "log_cnt_tgt", "log_demog_homeval", "log_int_tgt", "log_rfm1", "log_rfm10",
 "log_rfm2", "log_rfm3", "log_rfm4", "rfm5", "rfm6", "rfm7", "rfm8",
 "rfm9", "rfm11", "rfm12", "demog_age", "demog_inc", "demog_pr"
)

Store results
results <- lapply(variables_to_check, function(v) outlier_check_plot(bank,
 v))
names(results) <- variables_to_check

Extract outlier counts and plots
outlier_counts <- sapply(results, function(x) x$count)
outlier_plots <- lapply(results, function(x) x$plot)

Print summary sorted by outlier count
outlier_summary <- data.frame(Variable = names(outlier_counts), Outlier_Count
 = outlier_counts)
outlier_summary <- outlier_summary %>% arrange(desc(Outlier_Count))
print(outlier_summary)

Display boxplots in 2x2 grid layout
plot_chunks <- ceiling(length(outlier_plots) / 4)
for (i in 1:plot_chunks) {
 idx <- ((i - 1) * 4 + 1):min(i * 4, length(outlier_plots))
 grid.arrange(grobs = outlier_plots[idx], ncol = 2)
}

...

```

### ### Impact of Outliers and Strategies for Handling Them

Outliers can significantly affect the performance of predictive models. They can distort model parameters, increase variance, and reduce the ability of models to generalize well to unseen data. The strategies outlined below can help manage the influence of outliers:

#### #### \*\*Impact of Outliers\*\*:

- **Linear Models**: Can lead to biased coefficients and inflated standard errors.
- **Distance-based Algorithms**: Outliers influence distance calculations, affecting models like KNN and SVM.
- **Tree-based Models**: More robust but can still be affected by extreme values.

#### #### \*\*Strategies to Handle Outliers\*\*:

1. **Removing Outliers**:
  - Use **IQR** or **Z-score** methods to remove extreme values.
2. **Capping/Winsorization**:

- Limit extreme values to a certain percentile (e.g., 95th percentile).
- 3. **\*\*Log Transformation\*\***:
  - Apply log or Box-Cox transformations to reduce the spread of data and make models less sensitive to outliers (already applied to `cnt\_tgt`, `int\_tgt`, etc.).
- 4. **\*\*Robust Models\*\***:
  - Use **\*\*Ridge\*\***, **\*\*Lasso\*\***, or **\*\*Huber Regression\*\*** to reduce the impact of outliers on model training.
- 5. **\*\*Quantile Regression\*\***:
  - Focus on specific quantiles (e.g., 50th percentile) to minimize the impact of outliers.
- 6. **\*\*Outlier Detection\*\***:
  - Use algorithms like **\*\*Isolation Forest\*\*** to automatically identify and handle outliers.

#### #### **\*\*Strategy for This Dataset\*\***:

- **\*\*Log Transform\*\***: Continue using the log-transformed variables (e.g., `cnt\_tgt`, `rfm1`, `rfm2`).
- **\*\*Boxplots and IQR\*\***: Identify extreme outliers in both transformed and non-transformed variables.
- **\*\*Capping or Removal\*\***: Cap or remove extreme outliers where appropriate (e.g., `cnt\_tgt`).
- **\*\*Robust Models\*\***: Consider tree-based or regularized regression models to reduce the impact of outliers.

## # 6. Target Variable Analysis

```

```{r target variable analysis, echo=TRUE, warning=FALSE}
# indicate the target variables int_tgt, cnt_tgt, and b_tgt
# Create a bar plot for b_tgt with count labels and plain y-axis
b_tgt_plot <- ggplot(bank, aes(x = as.factor(b_tgt))) +
  geom_bar(fill = "#00AFBB", color = "black") +
  geom_text(stat = "count", aes(label = ..count..), vjust = -0.5, size = 4) +
  scale_y_continuous(labels = scales::comma) +
  labs(title = "Distribution of b_tgt", x = "b_tgt", y = "Count") +
  theme_minimal()

# Create histogram for int_tgt with colors and plain y-axis
int_tgt_hist <- ggplot(bank, aes(x = int_tgt)) +
  geom_histogram(bins = 30, fill = "#E7B800", color = "black") +
  scale_y_continuous(labels = scales::comma) +
  labs(title = "Histogram of int_tgt", x = "int_tgt", y = "Frequency") +
  theme_minimal()

# Create histogram for cnt_tgt with colors and plain y-axis
cnt_tgt_hist <- ggplot(bank, aes(x = cnt_tgt)) +
  geom_histogram(bins = 30, fill = "#FC4E07", color = "black") +
  scale_y_continuous(labels = scales::comma) +
  labs(title = "Histogram of cnt_tgt", x = "cnt_tgt", y = "Frequency") +
  theme_minimal()

# Create a list to store plots
plots <- list()

plots$b_tgt_plot <- b_tgt_plot
plots$int_tgt_hist <- int_tgt_hist
plots$cnt_tgt_hist <- cnt_tgt_hist

```

```

# Display the plots
plots$b_tgt_plot
plots$int_tgt_hist
plots$cnt_tgt_hist
```

```{r tgt_stats, echo=TRUE}
# Load gt
library(gt)

# Define target variables
tgt_vars <- c("int_tgt", "cnt_tgt")

# Function to compute summary statistics
compute_summary_statistics <- function(bank, target_vars) {
  summary_stats <- bank %>%
    dplyr::select(dplyr::all_of(target_vars)) %>%
    dplyr::summarise(dplyr::across(
      everything(),
      list(
        mean = ~mean(., na.rm = TRUE),
        sd = ~sd(., na.rm = TRUE),
        median = ~median(., na.rm = TRUE),
        min = ~min(., na.rm = TRUE),
        max = ~max(., na.rm = TRUE),
        skewness = ~e1071::skewness(., na.rm = TRUE)
      ),
      .names = "{col}_{fn}"
    )) %>%
    tidyr::pivot_longer(cols = everything(),
      names_to = c("Variable", "Statistic"),
      names_pattern = "(.*)_(.*)",
      values_to = "Value") %>%
    tidyr::pivot_wider(names_from = Statistic, values_from = Value) %>%
    dplyr::mutate(dplyr::across(where(is.numeric), ~round(., 2)))

  return(summary_stats)
}

# Run the function for target variables
summary_statistics_tgt <- compute_summary_statistics(bank, tgt_vars)

# Create a beautiful gt table
summary_statistics_tgt %>%
  gt() %>%
  fmt_number(
    columns = where(is.numeric),
    decimals = 2, # Keep 2 decimal places
    use_seps = TRUE # Add commas (thousand separators)
  ) %>%
  tab_header(
    title = "Summary Statistics for Target Variables"
  ) %>%
  tab_options(
    table.width = pct(100),
    table.font.size = 14,

```

```

    heading.align = "center",
    data_row.padding = px(6)
  )
}

```{r b_tgt_stats, echo=TRUE}
Function to compute summary statistics for a binary variable
compute_binary_summary <- function(data, binary_var) {
 binary_data <- data[[binary_var]]

 # Remove missing values
 binary_data <- binary_data[!is.na(binary_data)]

 total_n <- length(binary_data)
 n_ones <- sum(binary_data == 1)
 n_zeros <- sum(binary_data == 0)
 prop_ones <- round(n_ones / total_n, 2)
 prop_zeros <- round(n_zeros / total_n, 2)
 mode_val <- ifelse(n_ones >= n_zeros, 1, 0)

 summary_df <- data.frame(
 Variable = binary_var,
 Count = total_n,
 Count_1s = n_ones,
 Count_0s = n_zeros,
 Proportion_1s = prop_ones,
 Proportion_0s = prop_zeros,
 Mode = mode_val
)

 return(summary_df)
}

Run for binary variable b_tgt
binary_summary <- compute_binary_summary(bank, "b_tgt")

Display nicely formatted HTML table
binary_summary %>%
 kable("html", caption = "Summary Statistics for Binary Target Variable
 (b_tgt)") %>%
 kable_styling(full_width = FALSE, bootstrap_options = c("striped",
 "hover"))
```

```{r normality tests, echo=TRUE}
Load necessary library
library(nortest)

Formal Normality Testing (Large Sample Safe)
for (var in tgt_vars) {
 cat("\nAnderson-Darling Test for", var, "\n")
 print(ad.test(bank[[var]]))

 cat("\nKolmogorov-Smirnov Test for", var, "\n")
 # Standardize the variable first
 var_std <- scale(bank[[var]])

```

```
print(ks.test(var_std, "pnorm"))
}
```

```
```
```

Normality Tests for Target Variables

Anderson-Darling Test Results

```
- `int_tgt`:
  - Test Statistic: A = 4250.3
  - p-value < 2.2e-16
- `cnt_tgt`:
  - Test Statistic: A = 242695
  - p-value < 2.2e-16
```

Both tests strongly reject the null hypothesis of normality (p-value << 0.05), indicating that neither `int\_tgt` nor `cnt\_tgt` follows a normal distribution.

Kolmogorov-Smirnov Test Results

```
- `int_tgt`:
  - D = 0.11227
  - p-value < 2.2e-16
- `cnt_tgt`:
  - D = 0.47252
  - p-value < 2.2e-16
```

Similar to the Anderson-Darling test, the Kolmogorov-Smirnov test results indicate significant deviation from a normal distribution.

Note: The KS test assumes no ties in the data, and warnings were issued, but given the extremely small p-values, the evidence against normality remains very strong.

Interpretation and Impact on Modeling Choices

Given the strong evidence of non-normality for both `int\_tgt` and `cnt\_tgt`:

- Linear regression models may perform poorly unless transformations are applied.
- Transformations such as $\log(x + 1)$ or square root could be considered to reduce skewness if linear models are still desired.
- Alternatively, non-parametric or tree-based models (e.g., Random Forest, Gradient Boosting) can be applied, as they do not assume normality of the target.
- Performance metrics such as Mean Absolute Error (MAE) may be preferred over Mean Squared Error (MSE) to reduce the influence of extreme values (outliers).

Additionally:

- `cnt\_tgt`, being a count variable, may be better modeled with Poisson regression or Negative Binomial regression depending on dispersion.
- If `cnt\_tgt` has a high proportion of zeros, Zero-Inflated models could also be appropriate.

7. Predictor Analysis

```
```{r categorical predictor analysis, echo=TRUE}
```



```

List of categorical variable names excluding b_tgt
categorical_vars <- c("cat_input1", "cat_input2", "demog_ho", "demog_genf")

Function to compute frequency tables and create bar charts
compute_frequency_and_plot <- function(bank, categorical_vars) {
 # Loop through each variable
 lapply(categorical_vars, function(var) {
 # Frequency table for the variable
 freq_table <- bank %>%
 dplyr::select(dplyr::all_of(var)) %>%
 dplyr::filter(!is.na(.data[[var]])) %>%
 dplyr::group_by(.data[[var]]) %>%
 dplyr::summarise(Count = n(), .groups = "drop") %>%
 dplyr::mutate(
 Level = as.character(.data[[var]]), # Ensure Level is character
 Percent = round(100 * Count / sum(Count), 2),
 Variable = var
) %>%
 dplyr::select(Variable, Level, Count, Percent)

 # Display frequency table
 print(freq_table)

 # Plot bar chart for the variable
 ggplot(freq_table, aes(x = Level, y = Count, fill = Level)) +
 geom_bar(stat = "identity", show.legend = FALSE) +
 theme_minimal() +
 labs(title = paste("Frequency of", var), x = "Category", y = "Count") +
 theme(axis.text.x = element_text(angle = 45, hjust = 1))
 })
}

Run the function for categorical variables
compute_frequency_and_plot(bank, categorical_vars)

```

### ### Categorical Predictors Analysis

We analyzed the categorical predictors in the dataset to understand the number of categories each predictor has and how these categories are distributed. Below are the findings for each of the predictors:

- **\*\*cat\_input1 (Account Activity)\*\***
  - This variable has three categories:
    - **\*\*X\*\***: 831,371 observations (78.43%)
    - **\*\*Y\*\***: 77,847 observations (7.34%)
    - **\*\*Z\*\***: 150,820 observations (14.23%)
  - The majority of the observations (78.43%) are in the "X" category, indicating that most customers fall under this category. The "Y" category has a significantly smaller proportion (7.34%), while the "Z" category has 14.23% of the observations.
- **\*\*cat\_input2 (Customer Value Level)\*\***
  - This variable has five categories:
    - **\*\*A\*\***: 188,398 observations (17.77%)
    - **\*\*B\*\***: 192,382 observations (18.15%)
    - **\*\*C\*\***: 169,550 observations (15.99%)

- **\*\*D\*\***: 122,282 observations (11.54%)
- **\*\*E\*\***: 387,426 observations (36.55%)
- The distribution of the "cat\_input2" variable shows that the largest proportion of customers (36.55%) fall under the "E" category, followed by "B" (18.15%) and "A" (17.77%). Categories "C" and "D" have relatively smaller proportions, with "D" having the lowest proportion at 11.54%.
- **\*\*demog\_ho (Homeownership)\*\***
  - This binary variable indicates whether the customer is a homeowner:
    - **\*\*0 (Not Homeowner)\*\***: 476,741 observations (44.97%)
    - **\*\*1 (Homeowner)\*\***: 583,297 observations (55.03%)
  - The dataset is fairly evenly distributed between homeowners and non-homeowners, with a slightly higher percentage of homeowners (55.03%).
- **\*\*demog\_genf (Gender - Female)\*\***
  - This binary variable indicates the gender of the customer:
    - **\*\*0 (Not Female)\*\***: 464,288 observations (43.8%)
    - **\*\*1 (Female)\*\***: 595,750 observations (56.2%)
  - A majority of the customers (56.2%) are categorized as female, with 43.8% being categorized as not female.

### ### Numerical Predictors Analysis

#### ## Analysis of Numerical Predictors and Correlation Insights

##### ### 1. Range of Values for Numerical Predictors

The following table provides a summary of the range of values for the key numerical predictors in the dataset from prior code:

Variable	Range
<b>**int_tgt**</b>	0.00 to 500,000
<b>**cnt_tgt**</b>	0.00 to 6.00
<b>**rfm1**</b>	0.00 to 3,713.31
<b>**rfm2**</b>	1.58 to 650.00
<b>**rfm3**</b>	0.00 to 3,713.31
<b>**rfm4**</b>	0.00 to 10,000.00
<b>**rfm5**</b>	0.00 to 18.00
<b>**rfm6**</b>	0.00 to 127.00
<b>**rfm7**</b>	0.00 to 11.00
<b>**rfm8**</b>	0.00 to 46.00
<b>**rfm9**</b>	2.00 to 29.00
<b>**rfm10**</b>	0.00 to 77.00
<b>**rfm11**</b>	0.00 to 22.00
<b>**rfm12**</b>	0.00 to 571.00
<b>**demog_age**</b>	-1.00 to 89.00
<b>**demog_homeval**</b>	0.00 to 600,067.00
<b>**demog_inc**</b>	0.00 to 200,007.00

##### ### 2. Observations from Range Analysis

- **\*\*Wide Range of Values:\*\*** Variables such as **\*\*int\_tgt\*\***, **\*\*rfm1\*\***, **\*\*rfm3\*\***, **\*\*rfm4\*\***, **\*\*demog\_homeval\*\***, and **\*\*demog\_inc\*\*** exhibit very large ranges, suggesting a high degree of variability. This might

indicate the presence of extreme outliers that could skew analyses and should be checked further.

- **Narrower Range:** Variables like **cnt\_tgt** and **rfm5** have narrower ranges, indicating less variation and potential stability across customers. These could serve as stable predictors in models.
- **Data Quality Issues:** The variable **demog\_age** includes a negative value (-1.00), which likely represents missing or invalid data. This should be addressed during data cleaning.

---

### ### 3. Correlation Insights

The following table highlights some key correlation values between numerical predictors:

Variable1	Variable2	Correlation
rfm1	rfm2	0.91
rfm1	rfm3	0.96
rfm2	rfm3	0.88
rfm5	cnt_tgt	0.79
rfm6	rfm8	0.89

#### #### Correlation Observations:

- There is an **extremely high correlation** between **rfm1**, **rfm2**, and **rfm3**, with values reaching up to 0.96. This is expected because these variables represent different aspects of sales and promotional responses over time, leading to strong relationships between them.
- **rfm6** (Count Purchased Lifetime) and **rfm8** (Count Purchased Lifetime Dir Promo Resp) are also highly correlated (0.89), indicating that the total purchases and purchases via direct promotions are closely linked.
- **rfm5** (Count Purchased Past 3 Years) shows a moderate correlation with the target variable **cnt\_tgt** (0.79), making it a valuable predictor for modeling.

---

### ### 4. Multicollinearity Concerns

Given the **very strong correlations** observed between several predictors (e.g., **rfm1** and **rfm3**), we must be cautious about potential **multicollinearity**. Multicollinearity can distort model interpretations by inflating standard errors and making coefficient estimates unreliable.

#### #### Recommendations to Address Multicollinearity:

- **Variable Removal or Combination:** One approach is to remove one variable from each highly correlated pair (e.g., keep **rfm1** or **rfm3**, but not both). This reduces redundancy and helps stabilize the model.
- **Principal Component Analysis (PCA):** PCA can be used to reduce dimensionality while maintaining the variance across the correlated predictors.

- **Regularization Techniques:** Using techniques like **Ridge** or **Lasso regression** can help mitigate multicollinearity by shrinking the coefficients of less important variables.

---

### 5. Feature Selection

The strong correlation between some of the predictors and the target variable (**cnt\_tgt**) suggests that these features should be prioritized for model training. Notably, **rfm5** (Count Purchased Past 3 Years) shows a correlation of 0.79 with **cnt\_tgt**, making it a key predictor.

#### Prioritizing Strong Predictors:

- **rfm5** and **cnt\_tgt** should be retained in the feature set, as they have a strong relationship.
- Other variables with moderate to high correlations with the target, such as **rfm7** and **rfm12**, could also be considered for inclusion.

---

### 6. Modelling Considerations

When selecting models, it's important to consider how multicollinearity might impact performance, especially for linear models. **Tree-based models** like **Random Forest** or **Gradient Boosting Machines (GBM)** are robust to multicollinearity and can handle correlated features without significant issues.

However, if model interpretability is a priority, addressing multicollinearity should be part of the preprocessing steps to ensure clearer insights into predictor relationships and model coefficients.

## Impact of Predictor Characteristics on Feature Selection and Engineering

### 1. High Correlation Among RFM Variables

A major observation in the data is the **extremely high correlation** among several of the RFM variables, particularly **rfm1**, **rfm2**, and **rfm3**, with correlations ranging from 0.88 to 0.96. These variables are related to different aspects of sales and promotional response over time, leading to high multicollinearity.

#### Impact on Feature Selection:

- **Multicollinearity Issues:** High multicollinearity between these predictors can cause instability in linear models, where the model may struggle to differentiate between correlated variables, leading to inflated standard errors and less reliable coefficient estimates.
- **Feature Engineering Approach:** To mitigate this issue, we can:
  - **Remove Redundant Features:** Consider removing one of the highly correlated variables from each pair (e.g., choosing between **rfm1** or **rfm3**) to reduce redundancy.
  - **Dimensionality Reduction:** Use **Principal Component Analysis (PCA)** to transform these correlated variables into uncorrelated principal components, capturing most of the variance while eliminating multicollinearity.

- **Regularization Techniques:** If retaining multiple correlated variables is necessary for predictive power, using regularization techniques such as **Ridge Regression** or **Lasso** can help by penalizing large coefficients, thus reducing multicollinearity.

### 2. **Skewed Distributions in Certain Variables**

Several predictors, such as **int\_tgt**, **rfm1**, **rfm3**, **rfm4**, and **demog\_homeval**, exhibit skewed distributions with heavy tails (e.g., **int\_tgt** has a skewness of 13.30). This could indicate the presence of outliers or extreme values that may disproportionately influence model training.

#### **Impact on Feature Engineering:**

- **Handling Skewed Distributions:** For skewed variables, we can apply **log transformation** or **Box-Cox transformation** to stabilize variance and reduce the impact of outliers. This transformation can make these variables more normally distributed, improving the performance of certain models, particularly linear models and models that assume normality.
- **Outlier Detection and Treatment:** Identifying and addressing outliers in these variables (e.g., capping or winsorizing extreme values) can also prevent them from dominating the model's learning process.

### 3. **Numerical Predictors with Large Range of Values**

Some variables like **demog\_homeval** and **demog\_inc** have large ranges, spanning from zero to very large values. These wide ranges may lead to challenges in models that rely on scale-sensitive techniques (e.g., linear models, SVMs).

#### **Impact on Feature Engineering:**

- **Feature Scaling:** To address this, **standardization** (zero mean, unit variance) or **min-max scaling** (rescaling the range to [0, 1]) should be applied to these predictors. This will ensure that all variables contribute equally to the model and prevent variables with large ranges from dominating the learning process.
- **Log Transformation:** For financial variables like **demog\_homeval** and **demog\_inc**, log transformation can be particularly effective in compressing large values and reducing skewness.

### 4. **Target Variable Characteristics**

The target variable **cnt\_tgt** (Count of New Product Purchased) has a wide range but a relatively low mean and a moderate skewness (2.40). This suggests that most customers purchase a small number of products, but there are a few customers with very high counts.

#### **Impact on Feature Engineering:**

- **Target Transformation:** To deal with the skewness, we may consider applying a **log transformation** to **cnt\_tgt**, turning the skewed distribution into a more normal-like distribution. This transformation will help certain models, like linear regression or tree-based models, to better predict the target.
- **Handling Zero-Inflation:** Given that **cnt\_tgt** has many zero values, it might benefit from being treated as a **zero-inflated count variable**. Techniques like **Poisson regression** or **Negative**

Binomial regression\*\* could be considered for modeling this type of target variable.

### ### 5. \*\*Demographic Variables\*\*

Demographic variables like \*\*demog\_age\*\*, \*\*demog\_homeval\*\*, and \*\*demog\_inc\*\* have important implications for feature engineering. For example, \*\*demog\_age\*\* exhibits a moderate level of skewness and includes negative values (likely indicating missing or erroneous data), which suggests that this variable needs to be cleaned and transformed.

#### #### \*\*Impact on Feature Selection and Engineering:\*\*

- \*\*Data Cleaning:\*\* Any invalid or missing values in \*\*demog\_age\*\* (e.g., the negative value) should be addressed through imputation or removal, depending on the context and the percentage of missing data.
- \*\*Categorization:\*\* Age-related variables like \*\*demog\_age\*\* may also be grouped into \*\*age bands\*\* (e.g., 18-25, 26-35) to capture non-linear relationships between age and target behavior.
- \*\*Feature Interaction:\*\* Interaction terms between demographic variables and other predictors (e.g., \*\*demog\_inc\*\* with \*\*rfm5\*\*) could uncover valuable insights, especially in tree-based models, where interactions are automatically handled.

### ### 6. \*\*Targeted Feature Selection\*\*

Given the strong correlation between some of the RFM variables and the target variable \*\*cnt\_tgt\*\* (e.g., \*\*rfm5\*\* and \*\*cnt\_tgt\*\* have a correlation of 0.79), it is essential to focus on features that exhibit strong predictive power.

#### #### \*\*Impact on Feature Selection:\*\*

- \*\*Prioritize Strong Correlates:\*\* Variables like \*\*rfm5\*\* and other features that show strong relationships with the target (e.g., \*\*rfm12\*\*, \*\*rfm7\*\*) should be given priority during feature selection.
- \*\*Redundant Feature Removal:\*\* Features that are highly correlated with each other and with the target should be reviewed and potentially removed to avoid overfitting.

## Appendix.b.2

```

title: "Final Project"
author: "Shaun Levenson"
date: "`r Sys.Date()`"
output:
 html_document:
 toc: true
 toc_float: true
 self_contained: true
 code_folding: hide

Packages
```{r packages, echo=TRUE, message=FALSE, warning=FALSE}
options(timeout = 300)
options(java.parameters = "-Xmx8g")
# Install the required packages only if missing and suppress redundant output
packages <-
  c("h2o", "readr", "kableExtra", "dplyr", "ggplot2", "knitr", "tidyverse", "sca
    les", "gridExtra", "patchwork", "e1071", "mlbench", "tidymodels")
new_packages <- setdiff(packages, rownames(installed.packages()))
if (length(new_packages) > 0) install.packages(new_packages, quietly = TRUE)

# Load the required packages quietly
invisible(lapply(packages, function(pkg)
  suppressPackageStartupMessages(library(pkg, character.only = TRUE))))

invisible(capture.output(h2o.init()))

cat("Packages used:", packages, "\n")
```

```{r data, echo=TRUE}
load("F:/ECON/562_Analytics_2/Final Project/bank.Rda")

# Remove the columns that are not needed for the analysis
bank$account <- NULL
bank$dataset <- NULL
bank$demog_genm <- NULL

# Set cat_input1, cat_input2, and demog_genf as factors
bank[] <- lapply(bank, function(x) if(is.character(x)) as.factor(x) else x)
```

Missing Values, Outliers, and Data Cleaning

```{r missing, echo=TRUE}
# Check for missing values in the dataset
missing_values <- sapply(bank, function(x) sum(is.na(x)))
missing_percentage <- sapply(bank, function(x) sum(is.na(x)) / length(x) *
  100)
missing_summary <- data.frame(
  Variable = names(missing_values),
  Missing_Values = missing_values,
  Missing_Percentage = round(missing_percentage, 2)

```

```

)

# remove rows with 0 missing values
missing_summary <- missing_summary[missing_summary$Missing_Values > 0, ]
# Sort the summary by missing values
missing_summary <- missing_summary[order(-missing_summary$Missing_Values), ]

# remove row names
rownames(missing_summary) <- NULL

# Display the missing values summary in kable
kable(missing_summary, caption = "Missing Values Summary") %>%
  kable_styling(full_width = FALSE, position = "left") %>%
  column_spec(1, bold = TRUE) %>%
  column_spec(2, color = "blue") %>%
  column_spec(3, color = "red")

```

```

```

```

```

```{r missing_removal, echo=TRUE}
# Impute int_tgt missing values with 0
bank$int_tgt[is.na(bank$int_tgt)] <- 0

# Remove rows with missing values for demog_age and rfm3
bank <- bank[!is.na(bank$demog_age) & !is.na(bank$rfm3), ]

# Remove rows with missing values for cnt_tgt
bank <- bank[!is.na(bank$cnt_tgt), ]

```

```

```

```

There are 4 variables with missing values in the dataset. The variables are `int\_tgt`, `demog\_age`, `rfm3`, and `cnt\_tgt`. The percentage of missing values is also shown in the table.

#### **\*\*Missing Removal Reasoning\*\***

- `int\_tgt`: This variable is only set to `NA` when the customer is did not purchase a product (aka when b\_tgt is set = 0). We can impute this with `0` to indicate that they did not spend any money on purchasing a product.
- Additionally, only a small percentage of the data for `int\_tgt` is set to 0, so we can impute the missing values with `0` without losing too much information.
- For `demog\_age` and `rfm3`, I decided to exclude any observation with missing values. This is because the client is looking for a highly interpretable model, and I believe that using imputation methods will harm the model's interpretability.
- same for `cnt\_tgt` as well because there was only a single observation with a missing value.

As for `demog\_age`, I decided to remove any observations with an age of less than 16. This is because the client is a financial institution and including customers under the age of 16 will not be useful for the analysis due to legal reasons. Additionally, individuals under 16



```

 years old typically do not have financial authority or independence,
 making them less relevant for the analysis.
```{r skewness, echo=TRUE}
# Numerical variable names
numerical_vars <- c("int_tgt", "cnt_tgt", "rfm1", "rfm2", "rfm3", "rfm4",
                    "rfm5", "rfm6", "rfm7", "rfm8", "rfm9", "rfm10",
                    "rfm11", "rfm12", "demog_age", "demog_homeval",
                    "demog_inc", "demog_pr")

# Remove rows with age less than 16
bank <- bank[bank$demog_age >= 16, ]

# Function to compute summary statistics
compute_summary_statistics <- function(bank, numerical_vars) {
  summary_stats <- bank %>%
    dplyr::select(dplyr::all_of(numerical_vars)) %>%
    dplyr::summarise(dplyr::across(
      everything(),
      list(
        mean = ~mean(., na.rm = TRUE),
        sd = ~sd(., na.rm = TRUE),
        median = ~median(., na.rm = TRUE),
        min = ~min(., na.rm = TRUE),
        max = ~max(., na.rm = TRUE),
        skewness = ~e1071::skewness(., na.rm = TRUE)
      ),
      .names = "{col}_{fn}"
    )) %>%
    tidyr::pivot_longer(cols = everything(),
                        names_to = c("Variable", "Statistic"),
                        names_pattern = "(.*)_(.*)",
                        values_to = "Value") %>%
    tidyr::pivot_wider(names_from = Statistic, values_from = Value) %>%
    dplyr::mutate(dplyr::across(where(is.numeric), ~round(.x, 2)))

  return(summary_stats)
}

# Run the function
summary_statistics <- compute_summary_statistics(bank, numerical_vars)

# Display nicely formatted HTML table
summary_statistics %>%
  kable("html", caption = "Summary Statistics for Selected Numerical
    Variables") %>%
  kable_styling(full_width = TRUE, bootstrap_options = c("striped", "hover"))
  %>%
  scroll_box(width = "100%", height = "500px")
```
```{r categorical, echo=TRUE}
# List of categorical variable names
categorical_vars <- c("b_tgt", "cat_input1", "cat_input2", "demog_ho",
                     "demog_genf")

# Function to compute frequency tables
compute_frequency_tables <- function(bank, categorical_vars) {
  freq_tables <- lapply(categorical_vars, function(var) {

```

```

    bank %>%
      dplyr::select(dplyr::all_of(var)) %>%
      dplyr::filter(!is.na(.data[[var]])) %>%
      dplyr::group_by(.data[[var]]) %>%
      dplyr::summarise(Count = n(), .groups = "drop") %>%
      dplyr::mutate(
        Level = as.character(.data[[var]]), # Force Level to be character
        Percent = round(100 * Count / sum(Count), 2),
        Variable = var
      ) %>%
      dplyr::select(Variable, Level, Count, Percent)
  })

# Bind all tables into one
freq_tables_df <- dplyr::bind_rows(freq_tables)

return(freq_tables_df)
}

# Run the function
frequency_tables <- compute_frequency_tables(bank, categorical_vars)

# Display nicely formatted HTML table
frequency_tables %>%
  kable("html", caption = "Frequency Tables for Selected Categorical
    Variables") %>%
  kable_styling(full_width = TRUE, bootstrap_options = c("striped", "hover"))
  %>%
  scroll_box(width = "100%", height = "500px")
```



```

```{r transformations, echo=TRUE}
transform int_tgt, rfm1, rfm2, rfm3, rfm4, rfm10 using log(x+1)
 transformation, have the new variables named log_(x)
bank$log_int_tgt <- log(bank$int_tgt + 1)
bank$log_rfm1 <- log(bank$rfm1 + 1)
bank$log_rfm2 <- log(bank$rfm2 + 1)
bank$log_rfm3 <- log(bank$rfm3 + 1)
bank$log_rfm4 <- log(bank$rfm4 + 1)
bank$log_rfm10 <- log(bank$rfm10 + 1)

Create a new summary statistic data frame with the transformed variables
transformed_vars <- c("log_int_tgt", "log_rfm1", "log_rfm2", "log_rfm3",
 "log_rfm4", "log_rfm10")
transformed_summary_statistics <- compute_summary_statistics(bank,
 transformed_vars)
Display the summary statistics for transformed variables
transformed_summary_statistics %>%
 kable("html", caption = "Summary Statistics for Transformed Variables") %>%
 kable_styling(full_width = TRUE, bootstrap_options = c("striped", "hover"))
 %>%
 scroll_box(width = "100%", height = "500px")
```

```


Transformation Reasoning

- For each variable that was highly skewed, I applied a log transformation to reduce the skewness and make the distribution more normal. This is

```

important for many machine learning algorithms that assume normally distributed data.

- Additionally, using a log transformation allows for more interpretable coefficients in the models, as they can be interpreted as percentage changes.

```

```{r before and after, echo=TRUE}
vars <- c("int_tgt", "rfm1", "rfm2", "rfm3", "rfm4", "rfm10")

plot_list <- list()

for (v in vars) {
  p1 <- ggplot(bank, aes_string(x = v)) +
    geom_histogram(bins = 30, fill = "skyblue", color = "black") +
    labs(title = paste("Original", v), x = v, y = "Count") +
    theme_minimal()

  log_var <- paste0("log_", v)
  p2 <- ggplot(bank, aes_string(x = log_var)) +
    geom_histogram(bins = 30, fill = "salmon", color = "black") +
    labs(title = paste("Log-transformed", log_var), x = log_var, y = "Count")
    +
    theme_minimal()

  # Combine plots side by side
  plot_list[[v]] <- gridExtra::grid.arrange(p1, p2, ncol = 2)
}

# To display all plots, just run this:
# This will print each pair of plots one after another
for (p in plot_list) {
  print(p)
}
```

Interactions

```{r interactions, echo=TRUE}
# Create interaction terms
# Income and home value
bank$demog_inc_homeval <- bank$demog_inc * bank$demog_homeval

# Age and home value
bank$demog_age_homeval <- bank$demog_age * bank$demog_homeval

# Age and income
bank$demog_age_inc <- bank$demog_age * bank$demog_inc

```

Modelling by Target Variable

- `int_tgt`

```{r data_split, echo=TRUE}

```

```

options(h2o.max_mem_size = "8G")
#Import the dataset
bank_h2o <- as.h2o(bank)
options(timeout = 300)
# Split the data into training, validation, and test sets
splits <- h2o.splitFrame(data = bank_h2o, ratios = c(0.6, 0.2), seed = 123)
train <- splits[[1]]
valid <- splits[[2]]
test <- splits[[3]]
```

```r int_tgt, echo=TRUE}
# Set the response variable
response <- "log_int_tgt"

# Set the predictor variables
predictors_int <- c(
  "cat_input1", "cat_input2",
  "log_rfm1", "log_rfm2", "log_rfm3", "log_rfm4", "rfm5", "rfm6",
  "rfm7", "rfm8", "rfm9", "log_rfm10", "rfm11", "rfm12",
  "demog_age", "demog_genf", "demog_ho",
  "demog_homeval", "demog_inc", "demog_pr", "demog_inc_homeval",
  "demog_age_homeval", "demog_age_inc"
)
```

```r h2o_auto_int, echo=TRUE}
# Run AutoML with leaderboard on validation frame
automl <- h2o.automl(
  x = predictors_int,
  y = "log_int_tgt",
  training_frame = train,
  validation_frame = valid,
  leaderboard_frame = valid,
  max_runtime_secs = 300,
  seed = 123,
  stopping_rounds = 5,
  stopping_metric = "RMSE",
  nfolds = 0
)

# Convert leaderboard to data frame and get model types
leaderboard_df <- as.data.frame(automl@leaderboard)
leaderboard_df$model_id <- as.character(leaderboard_df$model_id)
leaderboard_df$model_type <- sapply(leaderboard_df$model_id, function(id) {
  model <- h2o.getModel(id)
  model@algorithm
})

# Keep only the top 4 unique model types
leaderboard_unique <- leaderboard_df %>%
  distinct(model_type, .keep_all = TRUE) %>%
  slice(1:4) # just in case fewer than 4

# Get model IDs and types for unique models
top_models <- leaderboard_unique$model_id
top_types <- leaderboard_unique$model_type

```

```

# Extract models
models <- lapply(top_models, h2o.getModel)

# --- VALIDATION METRICS (Leaderboard frame) ---
valid_perfs <- lapply(models, function(model) h2o.performance(model, newdata
  = valid))
valid_rmses <- sapply(valid_perfs, h2o.rmse)
valid_mses <- sapply(valid_perfs, h2o.mse)
valid_r2s <- sapply(valid_perfs, h2o.r2)

# Convert logged RMSE to unlogged scale
valid_unlogged_rmses <- sqrt(exp(valid_rmses^2) - 1)

validation_summary <- data.frame(
  Model = paste("Model", 1:length(top_models)),
  Type = top_types,
  RMSE = round(valid_rmses, 4),
  MSE = round(valid_mses, 4),
  Unlogged_RMSE = round(valid_unlogged_rmses, 2),
  R2 = round(valid_r2s, 4)
)

# Display validation summary table
kable(validation_summary, caption = "Validation Metrics: Top 4 Unique Model
  Types") %>%
  kable_styling(full_width = FALSE, position = "left") %>%
  column_spec(1, bold = TRUE) %>%
  column_spec(2, color = "purple") %>%
  column_spec(3, color = "blue") %>%
  column_spec(4, color = "red") %>%
  column_spec(5, color = "darkgreen")

# --- TEST METRICS (Only for the top model on leaderboard) ---
final_model <- models[[1]] # best model based on validation leaderboard

test_perf <- h2o.performance(final_model, newdata = test)
test_rmse <- h2o.rmse(test_perf)
test_mse <- h2o.mse(test_perf)
test_r2 <- h2o.r2(test_perf)
test_unlogged_rmse <- sqrt(exp(test_rmse^2) - 1)

test_summary <- data.frame(
  Model = "Final Model (Model 1)",
  Type = final_model@algorithm,
  RMSE = round(test_rmse, 4),
  MSE = round(test_mse, 4),
  Unlogged_RMSE = round(test_unlogged_rmse, 2),
  R2 = round(test_r2, 4)
)

# Display test summary table
kable(test_summary, caption = "Test Metrics: Final Selected Model") %>%
  kable_styling(full_width = FALSE, position = "left") %>%
  column_spec(1, bold = TRUE) %>%
  column_spec(2, color = "purple") %>%
  column_spec(3, color = "blue") %>%

```

```

column_spec(4, color = "red") %>%
column_spec(5, color = "darkgreen")

```

```{r int_tgt_plot, echo=TRUE}
# Variable importance for the final best model (models[[1]])
varimp_df <- as.data.frame(h2o.varimp(models[[1]]))

ggplot(varimp_df[1:20, ], aes(x = reorder(variable, -relative_importance), y
  = relative_importance)) +
  geom_bar(stat = "identity", fill = "steelblue") +
  coord_flip() +
  labs(title = "Top 20 Variable Importances (Final Best Model)",
        x = "Variable",
        y = "Relative Importance") +
  theme_minimal()

# Predictions on test set with final best model
best_model <- models[[1]]
predictions <- h2o.predict(best_model, newdata = test)
predictions_df <- as.data.frame(predictions)

# Actual values (log-transformed) from test set
actuals_log <- as.data.frame(test[["log_int_tgt"]])
actuals <- exp(actuals_log[[1]]) # back-transform

# Predicted values back-transformed
predicted <- exp(predictions_df[[1]])

# Actual vs Predicted dataframe
pred_vs_actual_df <- data.frame(
  Actual = actuals,
  Predicted = predicted
)

ggplot(pred_vs_actual_df, aes(x = Actual, y = Predicted)) +
  geom_point(color = "blue") +
  geom_abline(slope = 1, intercept = 0, color = "red") + # perfect
  prediction line
  labs(title = "Predicted vs Actual Values (Final Best Model)",
        x = "Actual Values",
        y = "Predicted Values") +
  theme_minimal() +
  theme(text = element_text(size = 12))

# GLM coefficient table only if the 4th model is a GLM
model_4 <- models[[4]] # 4th model from the unique top 4 list
if (model_4@algorithm == "glm") {
  coef_table <- model_4@model$coefficients_table
  coef_df <- coef_table[, c("names", "coefficients")]
  coef_df$Odds_Ratio_Percent <- round((exp(coef_df$coefficients) - 1) * 100,
    2)
  colnames(coef_df) <- c("Predictor", "Coefficient", "Odds Ratio (%)")

  kable(coef_df, caption = "GLM Coefficients and Odds Ratios (%) for 4th
    Model") %>%

```

```

    kable_styling(full_width = FALSE, position = "left") %>%
    column_spec(1, bold = TRUE) %>%
    column_spec(2, color = "blue") %>%
    column_spec(3, color = "red")

} else {
  cat("The 4th model is not a GLM. It is a", model_4@algorithm, "model.\n")
}
```

- `b_tgt`

```{r b_tgt, echo=TRUE}
# Set the response variable
response_b <- "b_tgt"

# Set the predictor variables
predictors_b <- c(
  "cat_input1", "cat_input2",
  "log_rfm1", "log_rfm2", "log_rfm3", "log_rfm4", "rfm5", "rfm6",
  "rfm7", "rfm8", "rfm9", "log_rfm10", "rfm11", "rfm12",
  "demog_age", "demog_genf", "demog_ho",
  "demog_homeval", "demog_inc", "demog_pr", "demog_inc_homeval",
  "demog_age_homeval", "demog_age_inc"
)
```

```{r h2o_auto_b, echo=TRUE}
# Run AutoML with validation leaderboard frame
automl_b <- h2o.automl(
  x = predictors_b,
  y = response_b,
  training_frame = train,
  validation_frame = valid,
  leaderboard_frame = valid, # validation used for leaderboard
  max_runtime_secs = 300,
  seed = 123,
  stopping_rounds = 5,
  stopping_metric = "AUC",
  nfolds = 0
)

# Convert leaderboard to data frame and get model types
leaderboard_df_b <- as.data.frame(automl_b@leaderboard)
leaderboard_df_b$model_id <- as.character(leaderboard_df_b$model_id)
leaderboard_df_b$model_type <- sapply(leaderboard_df_b$model_id, function(id)
  {
    model <- h2o.getModel(id)
    model@algorithm
  })

# Keep only the top 4 unique model types (from validation leaderboard)
leaderboard_unique_b <- leaderboard_df_b %>%
  distinct(model_type, .keep_all = TRUE) %>%
  slice(1:4)

top_models_b <- leaderboard_unique_b$model_id

```

```

top_types_b <- leaderboard_unique_b$model_type

# Extract the top 4 models
models_b <- lapply(top_models_b, h2o.getModel)

# Validation AUC for top 4 models (from leaderboard)
validation_performances_b <- lapply(models_b, function(model)
  h2o.performance(model, newdata = valid))
validation_auc_b <- sapply(validation_performances_b, h2o.auc)

# Build validation summary table
validation_summary_b <- data.frame(
  Model = paste("Model", 1:length(top_models_b)),
  Type = top_types_b,
  Validation_AUC = round(validation_auc_b, 4)
)

# Display validation summary table
kable(validation_summary_b, caption = "Top 4 Unique Model Types: Validation
  Performance Summary for b_tgt") %>%
  kable_styling(full_width = FALSE, position = "left") %>%
  column_spec(1, bold = TRUE) %>%
  column_spec(2, color = "purple") %>%
  column_spec(3, color = "blue")

# Now evaluate the best model (models_b[[1]]) on the test set for final
  performance
test_performance_b <- h2o.performance(models_b[[1]], newdata = test)
test_auc_b <- h2o.auc(test_performance_b)

# Build test summary table for final chosen model
test_summary_b <- data.frame(
  Model = "Best Model (Model 1)",
  Type = top_types_b[1],
  Test_AUC = round(test_auc_b, 4)
)

# Display test performance summary
kable(test_summary_b, caption = "Final Best Model Test Performance for
  b_tgt") %>%
  kable_styling(full_width = FALSE, position = "left") %>%
  column_spec(1, bold = TRUE) %>%
  column_spec(2, color = "purple") %>%
  column_spec(3, color = "blue")

# Save the final best model
h2o.saveModel(models_b[[1]], path = "F:/ECON/562_Analytics_2/Final
  Project/models/model_1_b_tgt", force = TRUE)

...

```{r b_tgt_plot, echo=TRUE}
=== Variable Importance for Best Model ===
varimp_df_b <- as.data.frame(h2o.varimp(models_b[[1]]))

Plot Top 20 Variable Importances

```



```

ggplot(varimp_df_b[1:20,], aes(x = reorder(variable, -relative_importance),
 y = relative_importance)) +
 geom_bar(stat = "identity", fill = "steelblue") +
 coord_flip() +
 labs(title = paste("Top 20 Variable Importances (", top_types_b[1], ")",
 sep = ""),
 x = "Variable",
 y = "Relative Importance") +
 theme_minimal()

=== ROC / AUC Curves for Best Model ===
perf_train <- h2o.performance(models_b[[1]], newdata = train)
perf_valid <- h2o.performance(models_b[[1]], newdata = valid)
perf_test <- h2o.performance(models_b[[1]], newdata = test)

Extract TPR and FPR
roc_train <- as.data.frame(h2o.tpr(perf_train))
roc_train$fpr <- as.data.frame(h2o.fpr(perf_train))$fpr

roc_valid <- as.data.frame(h2o.tpr(perf_valid))
roc_valid$fpr <- as.data.frame(h2o.fpr(perf_valid))$fpr

roc_test <- as.data.frame(h2o.tpr(perf_test))
roc_test$fpr <- as.data.frame(h2o.fpr(perf_test))$fpr

Plot
plot(roc_trainfpr, roc_traintpr, type = "l", col = "green", lwd = 2,
 xlab = "False Positive Rate", ylab = "True Positive Rate",
 main = "ROC Curves for Best Model")
lines(roc_validfpr, roc_validtpr, col = "blue", lwd = 2)
lines(roc_testfpr, roc_testtpr, col = "red", lwd = 2)
abline(a = 0, b = 1, lty = 2, col = "gray") # Diagonal reference

Add legend
legend("bottomright", legend = c("Train", "Validation", "Test"),
 col = c("green", "blue", "red"), lwd = 2, bty = "n")

=== GLM Coefficient Table (4th Model) ===
model_4 <- h2o.getModel(top_models_b[4])

if (model_4@algorithm == "glm") {
 coef_table <- model_4@model$coefficients_table

 coef_df <- coef_table[, c("names", "coefficients")]
 coef_df$`Odds Ratio (%)` <- round((exp(coef_df$coefficients) - 1) * 100, 2)

 colnames(coef_df) <- c("Predictor", "Coefficient", "Odds Ratio (%)")

 kable(coef_df, caption = "GLM Coefficients and Odds Ratios (%)") %>%
 kable_styling(full_width = FALSE, position = "left") %>%
 column_spec(1, bold = TRUE) %>%
 column_spec(2, color = "blue") %>%
 column_spec(3, color = "red")
} else {
 cat("The 4th model is not a GLM. It is a", model_4@algorithm, "model.\n")
}
` ``

```

```

```{r confusion_matrix, echo=TRUE}
# Get the best model (first in the list)
best_model <- models_b[[1]]
best_model_type <- top_types_b[1]

# Generate confusion matrix on test set
best_model_test_cm <- as.data.frame(h2o.confusionMatrix(best_model, newdata =
  test))

# Display as a styled table
library(knitr)
library(kableExtra)

kable(best_model_test_cm, caption = paste("Confusion Matrix on Test Set -",
  best_model_type, "(Best Model)")) %>%
  kable_styling(full_width = FALSE, position = "left") %>%
  column_spec(1, bold = TRUE) %>%
  column_spec(2, color = "red") %>%
  column_spec(3, color = "green") %>%
  column_spec(4, color = "blue") %>%
  row_spec(0, bold = TRUE, background = "#f2f2f2")

...

- `cnt_tgt`

```{r cnt_tgt, echo=TRUE}
Set the response variable
response_cnt <- "cnt_tgt"

Set the predictor variables
predictors_cnt <- c(
 "cat_input1", "cat_input2",
 "log_rfm1", "log_rfm2", "log_rfm3", "log_rfm4", "rfm5", "rfm6",
 "rfm7", "rfm8", "rfm9", "log_rfm10", "rfm11", "rfm12",
 "demog_age", "demog_genf", "demog_ho",
 "demog_homeval", "demog_inc", "demog_pr", "demog_inc_homeval",
 "demog_age_homeval", "demog_age_inc"
)

...

```{r h2o_auto_cnt, echo=TRUE}
# Run AutoML for count target (Poisson regression setup)
automl_cnt <- h2o.automl(
  x = predictors_cnt,
  y = response_cnt,
  training_frame = train,
  validation_frame = valid,
  leaderboard_frame = valid, # validation used for leaderboard
  max_runtime_secs = 300,
  seed = 123,
  stopping_rounds = 5,
  stopping_metric = "RMSE",
  nfolds = 0
)

```

```

# Convert leaderboard to data frame and get model types
leaderboard_df_cnt <- as.data.frame(automl_cnt@leaderboard)
leaderboard_df_cnt$model_id <- as.character(leaderboard_df_cnt$model_id)
leaderboard_df_cnt$model_type <- sapply(leaderboard_df_cnt$model_id,
  function(id) {
    model <- h2o.getModel(id)
    model@algorithm
  })

# Keep only the top 4 unique model types
leaderboard_unique_cnt <- leaderboard_df_cnt %>%
  distinct(model_type, .keep_all = TRUE) %>%
  slice(1:4)

# Get model IDs and types
top_models_cnt <- leaderboard_unique_cnt$model_id
top_types_cnt <- leaderboard_unique_cnt$model_type

# Extract top models
models_cnt <- lapply(top_models_cnt, h2o.getModel)

# Evaluate on validation (already used in leaderboard)
performances_cnt_valid <- lapply(models_cnt, function(model)
  h2o.performance(model, newdata = valid))
rmse_cnt_valid <- sapply(performances_cnt_valid, h2o.rmse)
mse_cnt_valid <- sapply(performances_cnt_valid, h2o.mse)

# Build validation summary table
summary_table_cnt <- data.frame(
  Model = paste("Model", 1:length(top_models_cnt)),
  Type = top_types_cnt,
  RMSE = round(rmse_cnt_valid, 4),
  MSE = round(mse_cnt_valid, 4)
)

# Display summary table for validation set
kable(summary_table_cnt, caption = "Top 4 Unique Model Types: Validation
  Performance Summary for cnt_tgt") %>%
  kable_styling(full_width = FALSE, position = "left") %>%
  column_spec(1, bold = TRUE) %>%
  column_spec(2, color = "purple") %>%
  column_spec(3, color = "blue") %>%
  column_spec(4, color = "red")

# Evaluate only the best model on the test set
best_model_cnt <- models_cnt[[1]]
perf_test_cnt <- h2o.performance(best_model_cnt, newdata = test)
rmse_test_cnt <- h2o.rmse(perf_test_cnt)
mse_test_cnt <- h2o.mse(perf_test_cnt)

# Display separate table for best model test performance
data.frame(
  Model = "Best Model (Test Set)",
  Type = top_types_cnt[1],
  RMSE = round(rmse_test_cnt, 4),
  MSE = round(mse_test_cnt, 4)
)

```

```

) %>%
  kable(caption = "Best Model Performance on Test Set (cnt_tgt)") %>%
  kable_styling(full_width = FALSE, position = "left") %>%
  column_spec(1, bold = TRUE, color = "darkgreen") %>%
  column_spec(2, color = "purple") %>%
  column_spec(3, color = "blue") %>%
  column_spec(4, color = "red")

# Save the best model
h2o.saveModel(best_model_cnt, path = "F:/ECON/562_Analytics_2/Final
  Project/models/model_1_cnt_tgt", force = TRUE)

```

```{r cnt_tgt_plot, echo=TRUE}
# Get variable importance for the best model
varimp_df_cnt <- as.data.frame(h2o.varimp(best_model_cnt))

# Plot top 20 variable importances using ggplot2
ggplot(varimp_df_cnt[1:20, ], aes(x = reorder(variable, -
  relative_importance), y = relative_importance)) +
  geom_bar(stat = "identity", fill = "steelblue") +
  coord_flip() +
  labs(title = "Top 20 Variable Importances (Best Model - cnt_tgt)",
    x = "Variable",
    y = "Relative Importance") +
  theme_minimal()

# Predicted vs Actual plot for the best model
predicted_cnt <- h2o.predict(best_model_cnt, newdata = test)
predicted_cnt_df <- as.data.frame(predicted_cnt)
predicted_cnt_df$actual <- as.data.frame(test)$cnt_tgt
predicted_cnt_df$actual <- as.numeric(predicted_cnt_df$actual)
predicted_cnt_df$predicted <- as.numeric(predicted_cnt_df$predicted)

ggplot(predicted_cnt_df, aes(x = actual, y = predicted)) +
  geom_point(alpha = 0.5, color = "darkorange") +
  geom_smooth(method = "lm", color = "blue") +
  labs(title = "Predicted vs Actual for cnt_tgt (Best Model)",
    x = "Actual cnt_tgt",
    y = "Predicted cnt_tgt") +
  theme_minimal()

# Inspect GLM coefficients if the 4th model is GLM
model_4_cnt <- h2o.getModel(top_models_cnt[4])
if (model_4_cnt@algorithm == "glm") {
  # Extract the full coefficient table
  coef_table_cnt <- model_4_cnt@model$coefficients_table

  # Keep relevant columns and calculate odds ratio percentage
  coef_df_cnt <- coef_table_cnt[, c("names", "coefficients")]
  coef_df_cnt$Odds_Ratio_Percent <- round((exp(coef_df_cnt$coefficients) - 1)
    * 100, 2)

  # Rename columns for clarity
  colnames(coef_df_cnt) <- c("Predictor", "Coefficient", "Odds Ratio (%)")

```

```
# Display table
kable(coef_df_cnt, caption = "GLM Coefficients and Odds Ratios (%) for
cnt_tgt") %>%
  kable_styling(full_width = FALSE, position = "left") %>%
  column_spec(1, bold = TRUE) %>%
  column_spec(2, color = "blue") %>%
  column_spec(3, color = "red")

} else {
  cat("The 4th model is not a GLM. It is a", model_4_cnt@algorithm,
      "model.\n")
}

` ``
```